

Technics SX-WSA1R

System Exclusive Reference

Acoustic Modeling Synthesizer Module

Applies equally to the SX-WSA1 keyboard

ROM set WSA-A 822 / WSA-C 230 / WSA-D 54

Unofficial

This document is not published, endorsed or checked by Technics or Panasonic. It was written by inspecting a disassembly of the program ROMs — the same ROMs ship in the SX-WSA1 keyboard and the SX-WSA1R rack — and checking the result against a bulk dump captured from a real instrument. The ROM set is the one the instrument's own ROM VERSION screen reports as the three numbers above.

Draft of version 1 — for evaluation by technical proof-readers

21 September 2026

How this was made. The research and the writing were carried out by the artificial-intelligence model **Claude Opus 5** (Anthropic), in its 1M-context configuration, working from the ROM images, a captured bulk dump and the published Technics manuals. The work was directed and coordinated by **Felipe Correa da Silva Sanches** `<juca@members.fsf.org>`.

Every measurement in this document is produced by a script committed beside it, so any figure here can be re-derived rather than taken on trust. Readers are asked to treat the whole of it as a draft under review: corrections are wanted, and the chapter on what this reference does not cover is part of what needs checking.

Contents

1	Overview	4
1.1	Applicability	4
1.2	Ports	4
1.3	Relationship to the published documentation	4
1.4	Verification	5
1.5	Conventions	6
2	At a glance	7
2.1	Frame	7
2.2	Messages	7
2.3	Rules	7
2.4	Bulk dump	8
2.5	Errors	8
3	Message framing	9
3.1	Identifiers	9
3.2	Termination	9
3.3	Length limit	9
4	The Technics message set	10
4.1	Fields	10
4.2	Commands	10
4.3	Which fields each message carries	11
4.4	Tempo	12
4.5	Parameter commands	12
5	Bulk dump	13
5.1	Categories and what each transmits	13
5.2	Data message — BTR	14
5.3	Payload encoding	15
5.4	Checksum	15
5.5	Requesting a dump — DRQ	15
5.6	Opening a transfer	16
5.7	Acknowledgement and retry	16
5.8	Continuation — CDD	17
5.9	Closing	17
5.10	Receiving	17

6	What a block contains	18
6.1	Signatures	18
6.2	Names	19
6.3	Inside the SOUND block	19
6.4	Inside the COMBINATION block	19
6.4.1	The tail of a part block	21
6.5	Inside the SEQUENCER block	22
6.5.1	Inside a song record	22
6.5.2	PERFORMANCE	24
7	Parameter messages	27
7.1	The address	27
7.2	Parts	27
7.3	What the instrument actually accepts	28
7.4	Parameters and the bulk dump are the same bytes	28
7.5	What a parameter accepts	29
7.6	Names	33
7.7	Whole-category request	33
7.8	The sound areas	33
8	The sound areas	34
8.1	How a normal sound is laid out	34
8.2	How a drum sound is laid out	35
8.3	What is transcribed here, and how far it is checked	35
8.4	Normal Sound parameters	36
8.5	Drum Sound parameters	40
9	Effects	45
9.1	What the type byte selects	45
9.2	What the value bytes mean	45
9.3	From a byte to a quantity	50
10	Enumerations	54
10.1	Control functions	54
10.2	Filter values	55
10.3	Interaction connect	55
11	Universal System Exclusive	56
11.1	Recognised messages	56
11.2	Messages not recognised	56
11.3	Practical consequences	56
11.4	The identifier is checked once	57
12	Settings that affect System Exclusive	58
12.1	The EXCLUSIVE filter	58
12.2	Which sockets carry what	59
12.3	Memory protect	59

13 Errors	60
13.1 The messages	60
13.2 ERROR 40 — wrong identification	60
13.3 ERROR 41 — reception	60
13.4 ERROR 42 — transmission	61
13.5 Interference from other traffic	61
14 SX-WSA1 and SX-WSA1R	62
15 Relationship to other Technics instruments	64
15.1 What is shared	64
15.2 What distinguishes the products	64
15.3 Dump regions differ	65
16 Worked examples	66
16.1 A parameter, written and read back	66
16.2 A frame from a real instrument	66
17 Scope of this edition	67
17.1 Deliberately outside it	67
17.2 What the guide names but does not pin down	67
17.3 Open, and what would close it	68

Chapter 1

Overview

This is unofficial documentation

Nothing here comes from Technics or Panasonic, and nothing here has been checked by them. This document was written by inspecting a disassembly of the program ROMs — the set the instrument’s own ROM VERSION screen reports as WSA-A 822, WSA-C 230 and WSA-D 54 — and by confronting what that showed with a bulk dump captured from a real instrument. The same ROMs ship in both machines, so it describes the SX-WSA1 keyboard as fully as the SX-WSA1R rack; chapter 14 gives the two behavioural differences between them.

Technics did publish a specification of its own, and where that specification covers the same ground this document follows it and uses its terms — section 1.3 says what it is and where it differs. Everything beyond it is inference from a disassembly, and inference can be wrong. Treat a statement here as a report of what one machine was observed to do, not as a guarantee about yours.

The Technics SX-WSA1R exchanges data over MIDI System Exclusive. This reference describes the messages it sends and accepts: their byte layout, the bulk-dump transfer protocol, the universal messages it honours, and what it reports when a transfer fails.

1.1 Applicability

This reference describes operating system version **2.0**. An earlier version 1 exists; its behaviour may differ and is not covered.

1.2 Ports

The instrument has two sets of MIDI sockets, MIDI 1 and MIDI 2. Ordinary performance data is handled on both. **Bulk dump transfers use MIDI 1 only** — a dump offered on MIDI 2 is not received, and the instrument sends dumps only from MIDI 1.

1.3 Relationship to the published documentation

Technics documented this instrument’s System Exclusive implementation, in the *Reference Guide* that shipped alongside the two user manuals. Its pages 34–35 carry a MIDI implementation chart headed *Synthesizer* [SX-WSA1] *Synthesizer module* [SX-WSA1R], and pages 41–56 give the message format, the address map and the parameter tables. Where that guide and

this reference describe the same thing, the guide’s terms are used here and its statements are followed.

This reference exists because the published specification describes the *format* and not the *behaviour*. It does not say what raises each error number, that a dump can be requested only while one screen is displayed, that the EXCLUSIVE filter governs a single message, that the rack refuses two functions silently, or that the instrument stops announcing its own General MIDI changes once a controller has sent one. Those are the subject of the chapters that follow, and they were established by decoding the program and by checking a dump from a real machine.

An earlier edition of this reference said no chart was published

It said the instrument shipped without a MIDI implementation chart. That was wrong: the chart and the full message format are in the *Reference Guide*, which is a separate volume from the user manual the claim had been checked against. The document now takes the published specification as its starting point.

One statement of the published documentation is contradicted here: the meaning of ERROR 42, set out in chapter 13.

1.4 Verification

The message layouts, the payload encoding and the checksum rule in this reference have been checked against a bulk dump captured from a real instrument: 2 883 messages. Despite its name the capture is a three-category session — SYSTEM, PART & MIDI, SOUND and COMBINATION — which matters, because the first of those is the category chapter 7 maps, so the parameter tables can be read against it directly. Of these, 2 876 carry a checksum, and all 2 876 verify under the rule given in chapter 5. Every transfer in the capture delivered exactly the byte count its header declared.

The parameter placement in chapter 7 has been checked against the same capture. The SYSTEM, PART & MIDI category was reassembled from it and each parameter’s own field read out of the result, for all 32 parts: 1 833 fields, of which 1 832 hold a value inside the range the instrument accepts for that parameter. Run against a plausible but wrong placement — one uniform stride for all 32 parts, instead of the two runs chapter 7 describes — the same check puts 69 fields out of range, so it is capable of rejecting a wrong map.

What the capture covers, and what covers the rest

Three of the four bulk-dump categories appear in it. SEQUENCER does not, and cannot: the capture came from a rack, which refuses that category (chapter 14).

That category is confirmed a different way. The eight address-and-length pairs the instrument accepts were decoded from the program, and Technics printed the same lengths in its *Reference Guide*. The two sets are identical, area by area, including all three SEQUENCER blocks and the guide's own word *Variable* for the last of them. The guide also prints SEQUENCER : WSA1 ONLY beside its request table — the same restriction the program gives the rack.

What a capture would still add for that one category is frame-level behaviour — its checksums, continuation and acknowledgements have been watched on the other three, and the program runs all four through one path — and the part of a song record that section 6.5 does not reach, which is now the 672 bytes outside its record stream and the PERFORMANCE block.

Two fields in that capture label themselves. Every part's MIDI channel holds its own part number, so the 32 parts spell out an unbroken ramp from 0 to 31 — which the wrong placement interrupts at part 8, the first part past the gap. Every part's pan holds 64, the value the instrument's own scale displays as CTR. Those two names in chapter 7 therefore rest on a real dump as well as on the instrument's screen.

1.5 Conventions

Byte values are hexadecimal. A field written *nn* takes one of the values listed for it. Ellipses inside a message stand for a body whose length depends on the message.

Every message begins with F0 and ends with F7, as required by MIDI.

Real-time messages are safe inside a transfer

System Real Time bytes (F8–FF) may be interleaved anywhere inside a System Exclusive message without disturbing it. Conversely, while the instrument is receiving a System Exclusive message it ignores F8 TIMING CLOCK, FA START, FB CONTINUE and FC STOP for transport purposes.

Chapter 2

At a glance

Everything in this chapter is stated again, with its conditions, in the chapters that follow. This page is for someone who already knows the protocol and wants the bytes.

2.1 Frame

F0 50 CMD 04 *md* 11 [ADR×3 SIZ×3 DT... *cn sm*] F7

04 is PC, the product; *md* is 00 for the SX-WSA1 and 01 for the SX-WSA1R, and both are accepted by both; 11 is VER, the exclusive specification's version. ADR and SIZ are 21-bit values in three seven-bit bytes, most significant first. There is no device or unit number anywhere.

2.2 Messages

F0 50 21 04 <i>md</i> 11 F7	HRQ	hand shake request
F0 50 22 04 <i>md</i> 11 F7	HRT	hand shake routine
F0 50 23 7E F7	ACK	acknowledge
F0 50 24 7E F7	NAK	not acknowledged
F0 50 25 <i>lo hi</i> F7	TMP	tempo, <i>lo</i> + 16 <i>hi</i> ; no checksum
F0 50 27 7E F7	EOK	end of category
F0 50 28 7E F7	END	end of dump
F0 50 29 7E F7	ERR	abort
F0 50 2A 7E F7	FUL	destination full
F0 50 2B ...	DRQ	request: ADR, SIZ, no DT
F0 50 2C ...	ITR	write one parameter
F0 50 2D ...	BTR	bulk data, first frame
F0 50 7E ...	CDD	bulk data, every frame after
F0 7E 7F 09 01 F7	General MIDI System On	
F0 7E 7F 09 02 F7	General MIDI System Off	

2.3 Rules

Payload. Each source byte goes out as two bytes, **high nibble first**: F1 becomes 0F 01. A payload is always an even number of bytes.

Checksum. $sm = (0 - S) \& 7F$, where S sums every byte from 50 through cn . Equivalently everything after F0, checksum included, sums to zero modulo 128. Verified on reception for 7E, 2B, 2C and 2D only.

Continue. cn is 01 while more frames follow, 00 on the last.

Size. At most 256 bytes per message including F0 and F7: 120 source bytes in a BTR frame, 125 in a CDD frame.

2.4 Bulk dump

ADR	area	menu name
40 00 00	PANEL	SYSTEM, PART & MIDI
20 00 00	SOUND MEMORY	SOUND
50 00 00	COMBINATION	COMBINATION
60 00 00	SEQUENCER	SEQUENCER (keyboard only)

Send DRQ with one of those to ask for a category. The instrument must be showing the SYSEX BULK DUMP screen to receive a dump, and bulk traffic works on the MIDI 1 sockets only.

2.5 Errors

ERROR 40 wrong product, model or version. ERROR 41 anything wrong with reception, including a missing acknowledgement. ERROR 42 an internal deadline, not a fault at the far end. ERROR 21 the destination is full.

Chapter 3

Message framing

3.1 Identifiers

The instrument accepts exactly two System Exclusive identifiers:

identifier	meaning
50	Matsushita (Technics)
7E	Universal Non-Real Time

A message carrying any other identifier is tracked to its F7 and discarded. No error is reported for it.

There is no device or unit number after the identifier

Many instruments place a device id immediately after the manufacturer identifier. The SX-WSA1R does not. The byte following 50 is already CMD, the command, described in chapter 4. A model byte does appear two positions later in most commands, but it names the model, not the device, and it is not where the convention would put a device id.

3.2 Termination

A message ends at F7. If any other status byte in the range 80–F6 arrives while a message is open, the message is abandoned and its contents discarded.

3.3 Length limit

A complete message may not exceed 256 bytes counting F0 and F7, that is 254 bytes between them. A longer message is abandoned and, inside a bulk-dump session, reported as a reception error. Dumps sent by the instrument use the full 256 bytes for all but the last message of each transfer.

Chapter 4

The Technics message set

4.1 Fields

Technics' published specification names the fields of a Technics System Exclusive message. This reference uses those names.

field	value	meaning
SOX	F0	exclusive status
IDC	50	Technics identifier
CMD		command, see below
PC	04	product category, WSA
	7E	DMY, the dummy used where no product is named
MD	00	model, SX-WSA1
	01	model, SX-WSA1R
VER	11	exclusive version 2.1
<i>then, for the messages that carry data:</i>		
ADR	3 bytes	a 21-bit address, seven bits each, most significant first
SIZ	3 bytes	length in source bytes, encoded the same way
DT		the data
CN	00/01	continue identifier
SM		checksum
EOX	F7	end of exclusive

VER is a version, not part of a model identifier

11 is the version of the *exclusive specification*, 2.1. It is not an operating system version — this instrument's operating system 2.0 sends VER = 11 — and it is not part of a model code. The model is MD, the byte before it.

4.2 Commands

After F0 50, CMD selects the message.

CMD		
21	HRQ	hand shake request
22	HRT	hand shake routine
23	ACK	acknowledge
24	NAK	negative acknowledge
25	TMP	tempo data
27	EOK	end of block
28	END	end
29	ERR	error
2A	FUL	memory full
2B	DRQ	data request
2C	ITR	individual data
2D	BTR	data block
7E	CDD	continuing data

The instrument accepts one further value that the published table does not list, 7F, and does not accept 26.

4.3 Which fields each message carries

ACK, NAK, EOK, END, ERR and FUL name no product: they carry PC = 7E DMY and stop there, with no MD and no VER. That is why they are five bytes long.

message	
F0 50 23 7E F7	ACK
F0 50 24 7E F7	NAK
F0 50 27 7E F7	EOK, end of block
F0 50 28 7E F7	END, end of dump
F0 50 29 7E F7	ERR
F0 50 2A 7E F7	FUL, memory full
F0 50 21 04 <i>md</i> 11 F7	HRQ, hand shake request
F0 50 22 04 <i>md</i> 11 F7	HRT, hand shake routine

DRQ, ITR and BTR carry PC, MD and VER, then ADR and SIZ, then CN and SM. ITR and BTR carry DT between them; DRQ, which asks rather than tells, carries none. CDD carries DT, CN and SM with no address at all.

MD is rewritten on transmission

The keyboard transmits MD = 00, the rack 01. The rack substitutes the byte as the message leaves, on every 21, 22, 2C and 2D message it sends, and recomputes the checksum afterwards, so the substitution is invisible to a receiver that validates it. A bulk dump captured from a real rack carries 04 01 11 throughout.

Reception is model-blind. Both values are accepted by both models, on every command that carries MD, and reach the same handler; a message is never refused, ignored or treated differently because of it. A receiver must therefore accept both, and a file dumped from one model loads into the other.

A message whose PC, MD or VER the instrument does not recognise is rejected with ERROR 40.

4.4 Tempo

TMP carries a tempo and nothing else:

F0 50 25 *lo hi* F7

Six bytes. Unlike every other Technics command it carries **no PC, no MD, no VER and no checksum**, and any bytes sent beyond the fifth are ignored. The tempo is nine bits sent low nibble first: $value = lo + 16 hi$. The accepted range is 40 to 300 beats per minute and a value outside it is treated as 120.

This message is the one thing the EXCLUSIVE filter governs; see chapter 12. It is also one of the two things the rack does not support; see chapter 14.

4.5 Parameter commands

DRQ requests and ITR writes an individual parameter. Both address the space chapter 7 sets out; chapter 16 works one through byte by byte.

Chapter 5

Bulk dump

Bulk dump transfers the instrument’s internal data as System Exclusive. The instrument is silent for the duration, and the transfer uses the MIDI 1 sockets only.

5.1 Categories and what each transmits

The SYSEX BULK DUMP screen offers five categories. Each sends one or more BTR messages at fixed addresses. The menu’s names and the published address map’s names for the same data differ; both are given.

category	area	ADR	SIZ	length	blocks
SYSTEM, PART & MIDI	PANEL	40 00 00	00 00 20	32	
		40 00 20	00 12 60	2 400	
SOUND	SOUND MEMORY	20 00 00	10 00 00	262 144	32
COMBINATION	COMBINATION	50 00 00	00 06 00	768	
		50 06 00	05 40 00	90 112	16
SEQUENCER	SEQUENCER	60 00 00	00 18 00	3 072	
		60 18 00	01 70 00	30 720	
		62 08 00	set at send time		
TOTAL KEYBOARD	the four categories in the order above				

Lengths are in source bytes; on the wire each occupies twice as many, because of the payload encoding described below.

These lengths are confirmed by the manufacturer

Every length in the table was decoded from the program, and Technics prints the same set in its *Reference Guide*. The two agree area by area, including SEQUENCER, whose last block the guide also calls variable rather than giving it a size.

The *blocks* column is internal structure, not extra messages: SOUND sends one header covering all 262 144 bytes, which is 32 repeats of an 8 192-byte layout, and COMBINATION’s second block one header covering 90 112 bytes as 16 repeats of 5 632. Those are the instrument’s 32 sounds and 16 combinations.

TOTAL KEYBOARD runs the four categories one after another in the order listed, each closed by its own end-of-category message, with a single end-of-dump message after the last.

The transmit order is not the menu order

The table above is in transmit order, which a capture from a real instrument confirms: SYSTEM, PART & MIDI goes first. The SYSEX BULK DUMP menu lists the same categories in a different order, with SOUND at the top. A receiver should take each transfer's ADR as the statement of what is arriving, and not infer the category from its position in the session.

The SEQUENCER category is the only one whose final message has no fixed length — it carries as much recorded material as the instrument holds.

The address is an exclusive address, not a memory address

ADR is a position in the published MIDI exclusive address map, which chapter 7 gives in full. It is not an address inside the instrument: SOUND and COMBINATION are held side by side in the machine, yet their exclusive addresses are 786 432 apart. For a bulk dump it is best treated as a fixed name. On reception the instrument compares ADR and SIZ against a fixed list and performs no arithmetic on them, so seven of the eight blocks are pinned on all six bytes; see below.

5.2 Data message — BTR

F0 50 2D 04 *nn* 11 *a2 a1 a0 l2 l1 l0 data... c sum* F7

nn MD, the model: 00 or 01.

a2 a1 a0 ADR, the destination, three bytes, most significant first, seven bits each: the value is $a_2 \times 128^2 + a_1 \times 128 + a_0$.

l2 l1 l0 SIZ, the length in *source* bytes, encoded the same way.

data DT, the payload, encoded as described below.

c CN, the continue identifier: 01 if further messages follow, 00 on the last.

sum SM, the checksum.

How much each frame carries

A complete message may not exceed 256 bytes including F0 and F7 (chapter 3), and the two frame shapes reach that limit differently, because a BTR header is eleven bytes where a CDD continuation header is two:

frame	total	payload bytes	source bytes
BTR, the first of a transfer	255	240	120
CDD, every one after it	256	250	125

Longer transfers are split across as many CDD frames as needed. Those figures are the observed maxima in a dump from a real instrument, not a stated limit; a sender may use less.

5.3 Payload encoding

Each source byte is sent as two message bytes, **high nibble first**:

$$b \longrightarrow (b \gg 4), \quad (b \& 0F)$$

So source byte **F1** appears on the wire as **0F 01**, and the payload occupies twice as many bytes as the length field states. A receiver rejoins them as $((\text{first} \& 0F) \ll 4) | (\text{second} \& 0F)$.

An odd number of payload bytes is invalid.

5.4 Checksum

Let S be the eight-bit sum of every byte from the identifier **50** through the continuation flag inclusive. Then

$$\text{sum} = (0 - S) \& 7F$$

The leading **F0**, the checksum byte itself and the trailing **F7** take no part in the sum. Equivalently, the sum of every byte after **F0** including the checksum is zero modulo 128.

The checksum is not verified for every family

On reception the checksum is checked only for families **7E**, **2B**, **2C** and **2D**. For the other accepted families a wrong checksum passes unremarked.

5.5 Requesting a dump — DRQ

A dump can be asked for over MIDI; the front-panel **SEND** button is not the only way to start one. The request names a category by the top byte of **ADR**:

F0 50 2B 04 nn 11 area 00 00 x x x F7

ADR	category requested
40	SYSTEM, PART & MIDI
20	SOUND
50	COMBINATION
60	SEQUENCER

The three bytes of **SIZ** are ignored; any values are accepted. The instrument then runs exactly the transfer the corresponding menu row would have run.

There is no request for **TOTAL KEYBOARD**: the four categories must be asked for one at a time.

A dump request is screen-gated; a parameter request is not

Both are DRQ, family 2B, and they do not behave alike. The routine that takes a request naming one of the four categories above tests the panel before anything else: it proceeds on the SYSEX BULK DUMP screen, or when two further panel bytes both read 01, and otherwise sets status 11.

That gate is not the whole story, and this edition cannot finish it. Against the emulated instrument a category request was refused with F0 50 29 7E F7 on the SYSEX BULK DUMP screen as well as at rest, and no request ever started a transfer. Something downstream of the panel test refuses as well, and what it is has not been isolated. Read the section above as the message the instrument parses, not as a transfer anyone has seen start this way.

A request naming a *parameter* (chapter 7) has no panel gate at all — nothing in the parameter engine reads the panel — and it does answer, at rest, both when the address is good and when it is not. That is the one 2B whose reply can be relied on.

5.6 Opening a transfer

A dump begins with a two-step exchange. Each step is attempted up to three times.

1. The sender transmits F0 50 21 04 *md* 11 F7 and expects F0 50 22 04 *nn* 11 F7.
2. The sender transmits F0 50 22 04 *md* 11 F7 and expects F0 50 23 7E F7.

MD is the sender's own: 00 from a keyboard and 01 from a rack, substituted on the way out as chapter 4 describes. A capture from a real rack carries 04 01 11 in both steps.

Read from the instrument's side, each step is gated twice: the panel must be on the SYSEX BULK DUMP screen, and the handshake state must be the one that step expects — zero for 21, one for 22. A 21 that arrives on any other screen, or a 22 before its 21, is answered with nothing at all. The state advances only on a step that passed both gates, so a sender that opens a transfer twice without closing the first will find its second 21 ignored.

If the exchange succeeds, every subsequent data message must be acknowledged. If all three attempts of *either* step fail, **the dump proceeds anyway, without acknowledgements**: both exits fall through to the same place with the acknowledgement bit still clear, and the transfer runs regardless.

The unacknowledged dump is paced by a fixed wait of the order of 50 ms between messages. The figure is given loosely on purpose: measured off the wire the gaps have a median of 45 ms, but the host timestamps jitter by as much as the quantity being measured, so only the order of magnitude survives. The 2.05 s the opening exchange waits per attempt is large enough to measure cleanly, and was: three attempts, 2.04 and 2.00 s apart.

5.7 Acknowledgement and retry

With acknowledgements in force, each data message must be answered with F0 50 23 7E F7. Any other reply causes the same message to be retransmitted, up to three attempts, after which the transfer is abandoned.

A reply is awaited for about 5.1 seconds; during the opening exchange the wait is about 2.05 seconds.

5.8 Continuation — CDD

Data messages after the first open with F0 50 7E, CDD, followed by DT, CN, SM and F7. A continuation carries no address.

5.9 Closing

At the end of each category the sender transmits F0 50 27 7E F7 and waits for an acknowledgement. At the end of the whole transfer it transmits F0 50 28 7E F7. If the transfer was abandoned after three failed retries it transmits F0 50 29 7E F7.

5.10 Receiving

When the SX-WSA1R is the receiver it answers each data message with F0 50 23 7E F7 to accept, F0 50 24 7E F7 to reject, and F0 50 2A 7E F7 when the incoming data is larger than the destination.

While a bulk-dump session is open the instrument accepts only the 50 identifier; universal messages are not honoured until the session ends.

The instrument must be on the SYSEX BULK DUMP screen to receive

A dump cannot be pushed to an idle instrument. Reception happens only while that screen is displayed; a dump sent at any other time is parsed, discarded, and not reported.

A dump offered on the MIDI 2 input is never received either, at any time. This is what lies behind the published instruction to use MIDI 1 for bulk transfers.

Seven of the eight patterns are pinned on all six bytes, address and length together. Sending a correct address with a shortened length is not a short transfer — it is an unrecognised message, refused with ERROR 41. Only SEQUENCER block 3 has its length read as a number, because that block's size varies. An arbitrary destination cannot be written.

The one length the message itself controls is SEQUENCER part 3, which is capped at 330 752 bytes; beyond that, and whenever the destination fills mid-message, the instrument answers F0 50 2A 7E F7 and reports ERROR 21.

Memory-full is treated as end-of-dump

When the SX-WSA1R is *sending* and the receiver answers F0 50 2A 7E F7 to report that its memory is full, the instrument treats that exactly as it treats F0 50 28 7E F7, end of dump. It stops, and it reports success. A sender that needs to know whether the data arrived cannot rely on the instrument to notice.

Chapter 6

What a block contains

Every block opens with a fixed signature, and two of them carry readable names, which let a program check a stored dump and list what is in it. Beyond that, all three of the SOUND, COMBINATION and SEQUENCER blocks are laid out below, each to a different depth.

6.1 Signatures

The bytes below are the first bytes of the reassembled block — after the payload has been rejoined from its nibbles, not as they appear on the wire. They were read out of a dump from a real instrument, and the two rows of each entry are the same sixteen bytes shown twice, as hexadecimal and as text.

block	ADR	first sixteen bytes	as text
SYSTEM, PART & MIDI 1	40 00 00	5A 5A 01 00 57 41 30 00 00 00 A0 00 00 00 00 00	ZZ..WA0.
SYSTEM, PART & MIDI 2	40 00 20	78 10 20 20 20 20 20 20 20 20 20 20 20 20 20 20	x.
SOUND	20 00 00	57 53 41 20 53 4F 55 4E 44 20 52 41 4D 20 53 30	WSA SOUN D RAM SO
COMBINATION 1	50 00 00	5A 5A 5A 5A 00 00 57 53 41 31 20 20 01 00 00 02	ZZZZ..WS A1
COMBINATION 2	50 06 00	78 10 20 53 70 61 63 65 20 49 6E 66 69 6E 69 74	x. Space Infinite

A file whose first bytes do not match the block it claims to be is not a dump of that category.

The first SYSTEM, PART & MIDI block is a signature in the fullest sense: all 32 of its bytes are a constant the instrument copies out of its own program when it initialises, with one byte filled in afterwards. It carries no settings.

The instrument writes its signature but does not check it

Two routines copy that constant into memory and both are plain copy loops. No comparison against it was found anywhere in either program image, and none against its leading 5A 5A either. **Do not expect a block with a wrong signature to be refused.** Checking it is something a librarian should do; the instrument does not.

The coverage of that negative: every three-byte pointer to the constant, which is two sites, and every immediate comparison against 5A 5A across both images. Code reaching the constant from a different base, or comparing through a pointer built elsewhere, would not have been seen.

6.2 Names

The SOUND block continues, immediately after its sixteen-byte signature, with sixteen-byte names in the instrument’s own character set, padded with spaces. The second COMBINATION block and the second SYSTEM, PART & MIDI block use a different framing: a two-byte prefix, then a sixteen-byte name, then two further bytes.

A librarian can therefore show the user what a stored file holds — “SWEEP PAD”, “Space Infinity” — without decoding anything else.

6.3 Inside the SOUND block

The SOUND block is sound *memory*, and it is laid out in four banks of 10000 bytes. Within a bank the stored sounds are packed back to back, and a stored sound **is the same 713-byte layout chapter 8 gives for the sound being edited** — so its name is its first sixteen bytes, and every parameter in that chapter is at its stated offset from the start of the record.

	at	
bank b	$10000 \times b$	four banks
stored sounds	359 in bank 0, 090 in the others	713 bytes each
a drum kit’s 128 notes	B468	150 bytes each

A stored drum kit is likewise the layout of chapter 8: its 128 note records sit at a 150-byte stride, each beginning with its thirteen-character name.

6.4 Inside the COMBINATION block

The larger COMBINATION block is **128 stored combinations of 704 bytes**, arranged as sixteen banks of eight. The banks’ names are the last 256 bytes of the smaller COMBINATION block, one name per bank; each combination carries its own sixteen-character name two bytes into its record.

A combination is a stream of parameter records

The 704 bytes are not a fixed layout but a **tagged stream**. Each record is a tag byte, a length byte, then that many bytes of payload; the stream ends with FF FF and fills the 704 exactly. Every one of the 128 combinations has the same 23 records in the same order.

at	tag	length	
000	78	16	the combination's name
012	60	12	effect common
020	61	30	EFFECT1
040	62	30	EFFECT2
060	63	30	REVERB
080	00, 20	30 each	part 1, two records
...	...		parts 2 to 8, the same pair
280	92	14	key scaling and scale tuning
290	79	44	part common
2BE	FF FF		end of stream

The tags are the instrument's own record numbers

This is what makes the format legible. A tag is not arbitrary: all 23 are record numbers in the parameter area of chapter 7, and a record's payload is that record's bytes. So 20 is part 1's own block — the same 20 that addresses part 1 in an ITR message — and the parameter tables name its fields directly.

The record numbers also run in address order with exactly two bytes between consecutive records, which are the tag and length. The stream is a copy of a region of the instrument's memory with its record headers intact.

One byte the parameter tables do not reach

A part's first record carries, at +15, six flags that no parameter addresses. Each says whether the part uses its own value for one of the MIDI MULTIPLE MESSAGES OUTPUT settings or the instrument's internal one:

bit	parameter
0	PROGRAM CHANGE
1	VOLUME
2	PANPOT
3	CHORUS DEPTH
4	REVERB DEPTH
5	BANK SELECT

Six sibling routines each test one of those bits and then choose between the part's internal field and its override field; the six parameters are exactly the six the published tables give an override at all. Bits 6 and 7 are not accounted for.

That also explains the second SYSTEM, PART & MIDI block, whose first bytes chapter 6 lists as 78 10: those are not a signature but the first record's tag and length — the same stream, beginning with the same name record.

How this was established, and why it is not a coincidence

A dump from a real instrument was walked at a 713-byte stride from the first stored sound. Sixty-four consecutive records began with a printable sixteen-character name, in every bank, and the drum runs gave 128 consecutive thirteen-character names in every bank. A stride one byte either side of 713 breaks the run at the first record.

The combination stream is checked differently: its records must tile the 704 bytes and land on FF FF. All 128 do, with the same 23 records in the same order, and all 23 tags are live record numbers in the parameter area.

6.4.1 The tail of a part block

Reserved. Block A holds the same value in all 3080 records at +02, +04, +10 and +15–+1A, and block B does at +00–+02, +0D, +11, +12 and +15. Every one reads zero. Write zero there.

A second program and bank. The last three bytes of each block are a second PROGRAM CHANGE & BANK for that part: +1B and +1C are one 7-bit number split four bits and three, and +1D is a bank drawn from the same values the part's own bank uses. Of the 890 parts carrying a non-zero one, the number and the bank *both* match the part's own in 296, and the number never matches without the bank matching too — against a control, the same test run against a neighbouring part, that lands 39 times. The pair +1B, +1C takes **exactly 128 distinct values** across the corpus, which is what a 7-bit number split four and three must and which no rival reading of those two bytes predicts. The shape is confirmed from outside the dump as well: the instrument's re-map files are sixteen groups of eight, the same 4+3 split, holding entries that are exactly (number, source) pairs. A second copy of an address that usually differs from the first is an indirection, which is also why the triple is not a function of the program number.

The program agrees. These bytes are never named by an address — a part record is reached through a pointer, which is why earlier searches for their addresses found nothing — but they are named by displacement off that pointer, and those are short distinctive byte strings. There are three sites that write all three bytes and three that read all three, in both images, and **not one site anywhere touches any of the three on its own**: they are a single field. At the writing site the values come back from a conversion whose input is written to a fixed address as *two* bytes and whose result is read from another as *three*, with an inverse beside it that takes three and returns two. A three-byte form of a two-byte quantity is exactly what a program number and a bank become.

Three bytes nothing reads. Block B's +18–+1A take fifteen distinct combinations across the corpus and read 01 01 01 or 00 01 01 in four records out of five. Nothing names them, and both halves of that have now been measured rather than assumed.

From the data: over all 3080 part records and 220 byte columns, none of the three is a copy of any other column, the three together are not a copy of any other three-byte run, and no column determines any of them — the best dependency is 0.62 of the target's entropy where a shuffled control scores 0.03 and a determination would score 1. The closest match, +1A agreeing with +03 in 97.6 % of records, is two nearly-constant columns agreeing and nothing more.

From the program: a part record is reached through a pointer, so these bytes are named by displacement, and there are two displacements they could carry — block B sits 20 past block A. Counting every individual access at either, across all four images and not only the two the instrument's panel runs on, there are 127. Three of them come near the pointer table

that holds part records, and all three are in one routine which indexes that table *without* the step that selects a part, so it reaches a sibling structure. None reaches a part record. The test for that is controlled: it fires on the bytes next door, where 6 of 91 accesses do reach a part record, including the sites that write the triple.

So these three bytes are carried in stored data and copied wholesale, and no code treats them as fields. For a librarian that is actionable — preserve them, do not compute them. What remains open is only what they are *for*, and since no code on this processor reads them the answer is not in this program; the obvious place to look next is the second processor.

Also open: which side of the indirection the triple holds — what the part asked for, or what it resolved to. Eight of the nine re-maps in the corpus are identity maps, so the two sides coincide wherever it could be checked. Deciding it needs a machine with a non-identity re-map loaded, dumped.

6.5 Inside the SEQUENCER block

No capture available for this reference contains a SEQUENCER transfer — the one to hand came from a rack, which refuses that category. The top-level arrangement is settled all the same, because the program gives it away, and the record’s own contents are read below from a native sequencer file off a disk, which carries the HEADER block byte for byte.

block	size	
LOCATION	3 072	one song record: the song being edited
HEADER	30 720	ten song records of 3 072
PERFORMANCE	variable	the recorded material

A song’s name is **six characters at offset CA** of its record. One routine establishes all of it at once: it blanks that name in three places, and its destinations are literal — the LOCATION block at **CA**, and the HEADER block at $\text{song} \times 3072 + \text{CA}$, the multiply written as a shift by eleven plus a shift by ten. Ten records of 3 072 fill the block exactly, and the published manual says the sequencer holds ten songs.

That LOCATION carries the name at the *same* offset is what shows it to be a song record as well, rather than a directory: it is the working copy, in the shape of a stored one.

6.5.1 Inside a song record

A native sequencer file from a disk, 01220497.SQF, is 30 720 bytes — the HEADER block exactly — and it decodes on the arrangement above without adjustment: ten records of 3 072, each opening **5A 5A 5A 5A**, each with six printable characters at **+CA**. Nine of the ten are byte-identical, which is what ten song slots with one song recorded into them should look like, and the recorded one differs from the empty template in 727 bytes.

Most of a song record is material this reference already names. From **+220** it is a tagged stream in the same form as a combination, {tag, length, payload} runs closed by **FF FF**, and it falls into three:

at	records	
+220–+4DF	23	a complete combination , 704 bytes
+4E0–+AFF	49	parts 8–31, then record 7A
+B00–+B7F	5	records 98, 99, 80, 91, 93

The first stream is a combination in the strict sense: the same 23 records in the same order and the same 704 bytes as section 6.4 gives, compared field for field rather than by eye. So a song carries its own combination inside it.

The part records then run 00–1F for block A and 20–3F for block B, complete and without gaps: **a song has thirty-two parts**, the same thirty-two the parameter grammar addresses at block 20 plus the part number. That is not only an inference from the tags — the owner’s manual advertises “32 MIDI parts available (32-part multi-timbral)”, and in the same breath that the sequencer records “up to 10 performances”, which is the ten song records this block is made of. Eight arrive inside the embedded combination and the other twenty-four follow it. Every one of their fields is named by chapter 7, because they are the same records — which puts 2 400 of a song record’s 3 072 bytes under names this reference already carries.

The 544 bytes ahead of the stream. Nine of the ten records are identical, so every byte a song writes is a byte where the recorded one differs, and there are 113 of them in this region. Read that way it lays out as follows. Fields marked *unused* = carry that value in an empty record.

at	size	
+000	4	signature 5A 5A 5A 5A
+004	1	01 in the recorded song, 00 in all nine empty ones
+023	16	a ramp 01–0F then 20
+034	16	the same ramp again
+07E	34	17 entries of 2 bytes, unused = FF FF
+0A0	17	17 entries of 1 byte, unused = 05
+0CA	6	the song NAME
+0D7	1	the TIME SIGNATURE numerator, as displayed — so <i>one more</i> than the conductor track’s 87 event carries
+100	51	17 entries of 3 bytes, unused = 00 FF FF; in use the first byte reads 80 and the other two are that track’s first block in PERFORMANCE
+134	204	51 entries of 4 bytes, identical in all ten records
+200	7	a second signature, 5A 5A 01 00 then WAO

The three tables are the part worth trusting, because they corroborate each other: they are at different offsets, they mark an unused entry three different ways, and all three are **seventeen** entries long. One such count would be an accident of where a reader chose to stop; three agreeing is a structure. In the recorded song every one of the seventeen +100 entries is in use, and the last ten of their 16-bit values run consecutively — which is what a block allocator handing out consecutive blocks looks like, though this reference does not decode what they index.

All three tables are per-track, and that is measured. The +100 table’s seventeen values are the seventeen chains of PERFORMANCE blocks, and following them produces seventeen *disjoint* chains, which nothing but a real per-track index would do. For the other two the program says so directly: the working copy of a song record sits at a fixed address, so each table has a fixed base, and one routine writes both of them from *one* track index — doubled for the two-byte table and left alone for the one-byte one.

Entry i is track i , and its events address part i . A controller event names the part it addresses in its second data byte, and for sixteen of the seventeen entries *every* such event in the chain names the entry’s own index — against a control, each entry paired with the next entry’s chain, that matches none of the seventeen. A chain’s last block is only partly used and its tail holds stale bytes, so it is excluded; with it left in, those stale events show up as the same handful of part numbers in every chain, which is how the tail was noticed.

The seventeenth is not a recording track, and the evidence is stronger than its not naming a part. Set its statuses against the other sixteen’s and the two sets are **disjoint in both directions**. Every one of the sixteen carries notes, controllers and program changes — 90, B0, B4, C0, D1, D2. Entry 16 carries none of those. It is 999 events of which **997 are the clock**, plus exactly two others, whose statuses — 80 and 87 — appear in none of the sixteen.

So the seventeen streams are the sixteen recording tracks Technics publishes plus one that only keeps time.

Its two events. 80 is the **tempo**. The playback routine dispatches on it, and its handler writes a 16-bit value into record 7A — passing that record’s own tag to the routine that publishes a record, which is how the record is identified rather than guessed — at +00, two bytes below the +02 the parameter map already names. A reset path writes the literal **120** to the same place, which is every sequencer’s default tempo. A display routine reads it as a *nine*-bit quantity and hands it to number formatting, and nine bits is what a tempo needs and a byte is not.

The transmitter for the event is in the program too, and it settles the encoding: it emits 80, the offset, then the nine bits as seven plus two. Running the handler’s arithmetic on the stored event and the transmitter’s arithmetic back again reproduces the stored bytes exactly, and the value between them is **150**.

87 is the **time signature**. Its handler puts a 7-bit value into a one-byte mailbox whose top bit means “changed”; the consumer clears that bit, *adds one*, and distributes the result, and the event’s own emitter — whose template carries the status 87 as a literal — subtracts one again. The setting it carries is cycled **1 to 8** with wraparound by a front-panel control. And a display routine loads the literal **4**, then that setting, and writes the pair: a numerator over a denominator of four, drawn immediately before the tempo. The screen’s own text reads TIME SIG = $n/4$.

So the stored value is one less than the numerator. This song carries 3, which is $4/4$, and the two events at the head of its conductor track are its time signature and its tempo.

The 128 bytes after it. +B80 to the end is identical in all ten records, recorded and empty alike, and is almost entirely zero. Nothing a song does touches it.

6.5.2 PERFORMANCE

The same disk set carries a companion file of 179 200 bytes, which is everything the fixed-size song records are not — so it is the recorded material, the PERFORMANCE block. That correspondence comes from how the disk splits the sequencer and not from anything in the program, and it is the one claim here that rests on an inference rather than a decode.

The area is blocks

PERFORMANCE is divided into **256-byte blocks numbered from one**. That is not inferred: the sequencer turns an address into a block number with three instructions — subtract the area’s base, shift right by eight, add one — and turns it back with a decrement and a shift left by eight.

A block’s first five bytes are a header. The **next block’s number is a 16-bit little-endian value at +03**, which is the offset the same routine reads it from, and a chain ends on 0000 or FFFF. The data is +05 to +FF.

The seventeen entries in a song’s header (section 6.5) hold each track’s *first* block. Following all seventeen gives 560 blocks and the chains are **disjoint** — no block belongs to two tracks — which is the first check that the reading is right. They account for four fifths of the area; the rest is free, and holds whatever was last written there.

A withdrawn hypothesis that was correct, and why the test missed it

An earlier edition of this chapter recorded, as a lead, that the seventeen values looked like 256-byte block numbers. It then reported a search for a link field — four block sizes, every offset within a block, both byte orders, requiring disjoint chains — that found nothing, and withdrew the lead on that basis.

The search was wrong, in one respect: it numbered blocks from zero. Every block it examined was the one before the block it meant. The hypothesis was right and the test that appeared to kill it was mis-specified, which is a more dangerous kind of error than a wrong guess, because a negative result looks like diligence.

What settled it was not a better search. It was reading the four instructions in the firmware that form a block number, which say +1 in as many words.

The events

Reassembled, a track is a flat stream with no delta before each event: a status byte at or above 80, then a fixed number of data bytes, every one below 80. Across all seventeen tracks that parses **41 646 events with not one byte left over**.

status	data	
81	0	advance the clock one step
90	5	a note
B0, B4	5	a controller
C0	5	a program change
D2	3	
E0	3	
D1	2	

Time is carried by the events themselves. 81 takes no data and does nothing but advance the clock, and it is much the commonest status — 19 980 of the 41 646. Every other event’s **first data byte is its offset inside the current step**: within a step those offsets never go backwards, 11 times in 15 306 consecutive pairs, where the same events shuffled go backwards 38.8 % of the time.

A note’s five bytes are then the offset, the note number, the velocity, and a gate length in two 7-bit halves. Over 14 380 notes the note-number column stays inside 25–126 and the velocity column inside 1–127, which is what those two columns have to look like and what no other assignment of the five produces.

A handful of statuses outside the table appear a few times each, at the ends of chains where a block’s last event is cut short; they are not part of the format.

Reassembly is self-checking

A block reassembled correctly comes out at exactly the length its own header declared. A wrong nibble order, or a payload taken with its trailing flag and checksum still attached, does not. Checking the length is a cheap way to confirm a receiver is unpacking correctly before trusting anything it produces.

Chapter 7

Parameter messages

Two commands address individual parameters rather than transferring blocks. ITR (2C) writes one; DRQ (2B) requests one.

F0 50 2B 04 *md* 11 *a2 a1 a0 s2 s1 s0 cn sm* F7

7.1 The address

ADR is a single 21-bit address, sent as three seven-bit bytes, most significant first. It is the same address space the bulk dump uses — a dump and a parameter write are two ways of reaching one map. The published map divides it as follows.

ADR	address	area	subarea	
00 00 00	000000	SYSTEM		real time
00 08 00	000400	SYSTEM		non-real time
00 10 00	000800	PART	COMMON	real time
00 18 00	000C00	PART	COMMON	non-real time
00 20 00	001000	PART	INDIVIDUAL	part 1, real time
00 20 40	001040	PART	INDIVIDUAL	part 1, non-real time
00 3F 40	001FC0	PART	INDIVIDUAL	part 32, non-real time
00 60 00	003000	PART	SPECIAL	real time
00 68 00	003400	PART	SPECIAL	non-real time
10 00 00	040000	NORMAL SOUND	INDIVIDUAL	real time
18 00 00	060000	DRUM SOUND	INDIVIDUAL	real time
20 00 00	080000	SOUND MEMORY	TOTAL	
40 00 00	100000	PANEL	HEADER	
40 00 20	100020	PANEL	PANEL DATA	
50 00 00	140000	COMBINATION	HEADER	
60 00 00	180000	SEQUENCER	LOCATION	

The last five are the bulk-dump categories of chapter 5. The ones above them are what an individual parameter message reaches.

7.2 Parts

A part occupies 80 bytes of the map, split in half: the first 40 are its real-time parameters and the second 40 its non-real-time ones. So for part *n*, counting from 1, the middle byte of

ADR is $20 + n - 1$, and the last byte is the parameter, below 40 for a real-time parameter and 40 or above for a non-real-time one.

All 32 parts take the same 57 parameters.

7.3 What the instrument actually accepts

Not every address in the map is a parameter. In the 00 area the instrument accepts 108 distinct addresses, which the tables below enumerate in full.

ADR middle	area	subarea	parameters
00, 01	SYSTEM		22
08	SYSTEM	non-real time	4
10, 11	PART	COMMON	24
20–3F	PART	INDIVIDUAL	57 each
60	PART	SPECIAL	1

Under 11, values of the last byte from 03 upward are admitted in three runs, one per effect processor:

run	processor	
20–36	EFFECT1	type, twenty values, send, dynamic control
40–56	EFFECT2	type, twenty values, send, dynamic control
60–75	REVERB	type, twenty values, dynamic control

Within a run the first address is the effect type and the next twenty are its parameters, whose meaning depends on the type selected. Any value of the last byte outside the three runs is accepted by the parser and then discarded, producing no error and no effect.

7.4 Parameters and the bulk dump are the same bytes

The PART and SYSTEM parameters and the PANEL bulk dump — the category the menu calls SYSTEM, PART & MIDI — are two views of one object. That dump carries 100000 to 100980, and every parameter of the 00 area is one byte, or a field of one byte, inside it.

This matters in both directions. An editor that writes parameters and a librarian that restores dumps are competing for the same storage, so a dump restored after a parameter write discards that write. It also gives a way to confirm any row of the tables below without opening the instrument: dump the category, change the one parameter, dump again, and the two dumps differ at the stated address and nowhere else.

Where a part's record sits

Each part occupies a 40-byte record in that dump, and the offsets in table 7.1 are relative to the start of it. The records are in two runs:

part	record begins at
1 to 8	$1000A2 + 40 \times (\text{part} - 1)$
9 to 32	$1002E2 + 40 \times (\text{part} - 9)$

The part records are not one array

The obvious formula, $1000A2 + 40 \times (\text{part} - 1)$, is correct for the first eight parts and wrong for the other twenty-four. A 40-byte gap after part 8 holds common-block storage, and the parts resume above it. Code that assumes a single stride will read and write the wrong part for parts 9 upward, and will corrupt two common blocks on the way past.

7.5 What a parameter accepts

Each parameter has a fixed data length and a range of accepted values. A value outside the range is rejected without an error message. The *bits* column gives which bits of the dump byte belong to the parameter; where that is not all of them, other parameters share the byte, and a dump editor changing one must preserve the rest.

Three parameters accept an enumerated set rather than a range; sending any other value has no effect.

Two parameters where the published range and the range check disagree

The *values* column below is the range the instrument's own check enforces. For all but two parameters that is also the published range. The exceptions:

00 00 08 **MASTER TUNING**. Published as C0–00–3F, a signed value with 00 at concert pitch, spanning 427.3 to 453.0 Hz. The range check admits 48 to 192. A dump from a real machine holds 00 there — the published centre, and not a member of the checked range.

00 *nn* 05 **EFFECT2 ON/OFF**. Published as 00–01. The range check admits 0 to 127, and the instrument passes the value through a substitution before storing it, so reading it back does not return what was written.

Send what the published specification gives.

The accepted values are what a write is checked against

They are what the instrument takes from an ITR, not a guarantee about what a stored byte contains. A librarian should not validate a dump by checking it against these ranges.

Each MAIN OUT EQUALIZER band is one two-byte field

Its four parameters do not occupy four bytes one each. A band is a 16-bit **little-endian** word: GAIN in bits 0–5, and in bits 6–10 a five-bit *index* into the 27-entry frequency table of section 9.3, so the index takes two bits from the low byte and three from the high one. The low band’s word begins at 1002B3 and the high band’s at 1002B5. That is why the two frequencies have no byte of their own to be listed against, and it is what the published ranges are already telling you: 0–17 and 17–26 are two sub-ranges of one 27-entry table, overlapping at 17.

The placement is not a reading of the bytes that happens to work. Taking 385 stored combinations from three sources gives 770 band observations, and of every way to put a contiguous six-bit gain and a contiguous five-bit index inside two bytes, in either byte order, **exactly one** places all 770 inside their declared ranges. The factory setting decodes to gain 24 of 0–48 — centre — in both bands, with index 11 low and 17 high; the one instrument in the corpus that had moved its low band reads gain 18, index 7.

One disagreement. The guide’s packed-field note for its pre/post equaliser format gives the five-bit index as three bits in the first byte and two in the second. For MAIN OUT EQUALIZER it is the other way round — that layout is inside the search above, and it fails.

Table 7.1: The 57 parameters of a part block. *p* is the third address byte; the offset is the byte’s position inside that part’s record in a SYSTEM, PART & MIDI bulk dump.

<i>p</i>	data	offset	bits	values	name
00	3	00	all	0–127	PROGRAM CHANGE & BANK
03	1	03	7F	0–127	VOLUME
04	1	05	7F	0–127	EFFECT1 SEND
05	1	06	7F	0–127	EFFECT2 ON/OFF
06	1	07	7F	0–127	REVERB SEND
07	1	08	7F	0–127	PANPOT
08	1	09	7F	28–100	KEY SHIFT
09	1	0A	all	0–255	FINE TUNE
0A	1	0B	7F	0–12	PITCH BEND RANGE
10	1	26	0F	0–1	ASSIGN MODE
14	1	0C	08	0–1	KEY SCALING
20	1	25	all	0–48	VELOCITY OFFSET
28	1	27	7F	0–127	KEY LAYER LOW
29	1	28	7F	0–127	KEY LAYER HIGH
2A	1	29	7F	0–127	VELOCITY LAYER LOW
2B	1	2A	7F	0–127	VELOCITY LAYER HIGH
30	1	23	all	0–1	MAIN OUT
31	1	24	all	enumerated, 4 values	SUB OUT
40	1	0D	20	0–1	LOCAL CONTROL
41	1	0D	1F	0–31	BASIC CHANNEL
42	1	0D	40	0–1	MIDI OUT SETTING
43	1	0D	80	0–1	MIDI IN SETTING
48	1	2F	10	0–1	MIDI INPUT FILTER: PROGRAM CHANGE
49	1	30	80	0–1	MIDI INPUT FILTER: BANK SELECT
4A	1	30	04	0–1	MIDI INPUT FILTER: VOLUME
50	1	33	10	0–1	MIDI OUTPUT FILTER: PROGRAM CHANGE

<i>p</i>	data	offset	bits	values	name
51	1	34	80	0–1	MIDI OUTPUT FILTER: BANK SELECT
52	1	34	04	0–1	MIDI OUTPUT FILTER: VOLUME
53	1	34	10	0–1	MIDI OUTPUT FILTER: PANPOT
54	1	34	20	0–1	MIDI OUTPUT FILTER: EFFECT DEPTH
55	1	33	40	0–1	MIDI OUTPUT FILTER: PITCH BEND
56	1	34	02	0–1	MIDI OUTPUT FILTER: MODULATION1
57	1	36	10	0–1	MIDI OUTPUT FILTER: MODULATION2
58	1	36	01	0–1	MIDI OUTPUT FILTER: REAL-TIME CREATOR-X
59	1	36	02	0–1	MIDI OUTPUT FILTER: REAL-TIME CREATOR-Y
5A	1	36	04	0–1	MIDI OUTPUT FILTER: REAL-TIME CONTROLLER-X
5B	1	36	08	0–1	MIDI OUTPUT FILTER: REAL-TIME CONTROLLER-Y
5C	1	34	01	0–1	MIDI OUTPUT FILTER: HOLD1
5D	1	36	20	0–1	MIDI OUTPUT FILTER: CONTROL PEDAL
5E	1	33	20	0–1	MIDI OUTPUT FILTER: AFTER TOUCH
60	1	0E	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: PROGRAM CHANGE
61	2	—	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: BANK SELECT
63	1	11	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: VOLUME
64	1	12	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: PANPOT
65	1	14	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: REVERB DEPTH
66	1	13	all	0–129	MIDI MULTIPLE MESSAGES OUTPUT: CHORUS DEPTH
68	1	37	all	28–100	MIDI OUT KEY TRANSPOSE
70	1	2B	40	0–1	CONTROLLER INTERNAL FILTER: PITCH BEND
71	1	2C	02	0–1	CONTROLLER INTERNAL FILTER: MODULATION1
72	1	2E	10	0–1	CONTROLLER INTERNAL FILTER: MODULATION2
73	1	2E	01	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CREATOR-X
74	1	2E	02	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CREATOR-Y
75	1	2E	04	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CONTROLLER-X
76	1	2E	08	0–1	CONTROLLER INTERNAL FILTER: REAL-TIME CONTROLLER-Y
77	1	2C	01	0–1	CONTROLLER INTERNAL FILTER: HOLD1
78	1	2E	20	0–1	CONTROLLER INTERNAL FILTER: CONTROL PEDAL
79	1	2B	20	0–1	CONTROLLER INTERNAL FILTER: AFTER TOUCH

Table 7.2: The parameters of the common blocks. The address is the one a SYSTEM, PART & MIDI bulk dump carries for the same byte. *command* marks an address that is an action rather than stored state, so no dump carries it; *outside* marks a byte that falls outside the block; -- marks a position this reference has not established.

<i>s</i>	<i>p</i>	data	address	bits	values	name
00	00	1	outside	01	0–1	SOUND MEMORY PROTECT
00	01	1	outside	02	0–1	COMBINATION MEMORY PROTECT
00	08	1	10094A	all	48–192	MASTER TUNING
00	10	1	10095A	0F	0–9	VELOCITY CURVE
00	11	1	10095C	7F	0–127	VELOCITY OFFSET
00	12	1	10095F	all	0–10	AFTER TOUCH CURVE
00	13	1	100960	7F	0–127	AFTER TOUCH THRESHOLD
00	20	1	10094E	all	enumerated, 4 values	DRUMS MAP SELECT
00	30	1	1008E4	0F	0–2	DATA LOAD FILTER: EFFECT & OUTPUT
00	31	1	1008E4	F0	0–2	DATA LOAD FILTER: REAL-TIME CREATOR 1-6
00	32	1	100907	04	0–1	DATA LOAD FILTER: COMBINATION OCTAVE
00	33	1	100907	02	0–1	DATA LOAD FILTER: MIDI SETTING
00	34	1	100907	01	0–1	DATA LOAD FILTER: KEY & VELOCITY LAYER
00	35	1	100907	08	0–1	DATA LOAD FILTER: MAIN OUT EQUALIZER
00	36	1	100907	10	0–1	DATA LOAD FILTER: KEY SCALING
00	40	2	—	7F	0–127	COMBINATION NUMBER and BANK
01	00	1	100935	0F	0–2	MIDI INPUT MODE
01	01	1	100935	F0	0–1	MIDI OUTPUT MODE
01	02	1	100936	1F	0–15	SINGLE CHANNEL
01	03	1	100936	20	0–1	LOCAL TOTAL
01	04	1	100932	03	0–1	PROGRAM CHANGE MODE
01	05	1	100932	08	0–1	SINGLE CHANNEL PROGRAM CHANGE
08	00	1	command	—	0–6	INITIAL
08	08	1	command	F0	0–1	PLAY MODE REQUEST
08	10	1	command	—	0–127	COMBINATION WRITE REQUEST
08	11	3	command	—	0–1	SOUND WRITE REQUEST
10	00	1	1002B2	7F	28–100	KEY TRANSPOSE
10	10	1	1002A3	80	0–1	KEY SCALING MODE
10	11	1	1002A2	all	enumerated, 15 values	TOTAL KEY SCALING
10	12	1	1002A3	0F	0–11	KEY SCALING SHIFT
10	13	1	1002A4	all	0–255	Ckey DETUNE SETTING
10	14	1	1002A5	all	0–255	Dbkey DETUNE SETTING
10	15	1	1002A6	all	0–255	Dkey DETUNE SETTING
10	16	1	1002A7	all	0–255	Ebkey DETUNE SETTING
10	17	1	1002A8	all	0–255	Ekey DETUNE SETTING
10	18	1	1002A9	all	0–255	Fkey DETUNE SETTING
10	19	1	1002AA	all	0–255	Gbkey DETUNE SETTING

<i>s</i>	<i>p</i>	data	address	bits	values	name
10	1A	1	1002AB	all	0–255	Gkey DETUNE SETTING
10	1B	1	1002AC	all	0–255	Abkey DETUNE SETTING
10	1C	1	1002AD	all	0–255	Akey DETUNE SETTING
10	1D	1	1002AE	all	0–255	Bbkey DETUNE SETTING
10	1E	1	1002AF	all	0–255	Bkey DETUNE SETTING
10	20	1	1002B3	6–10	0–17	MAIN OUT EQUALIZER LOW-FREQ
10	21	1	1002B3	3F	0–48	MAIN OUT EQUALIZER LOW-GAIN
10	22	1	1002B5	6–10	17–26	MAIN OUT EQUALIZER HIGH-FREQ
10	23	1	1002B5	3F	0–48	MAIN OUT EQUALIZER HIGH-GAIN
11	00	1	100034	01	0–1	EFFECT COMMON ALGORITHM
11	01	1	1002B7	0F	1–4	EFFECT1 OUTPUT SELECT
11	02	1	1002B7	F0	1–4	EFFECT2 OUTPUT SELECT
11	20	1	—	—	0–0	EFFECT1 TYPE
60	00	1	100905	7F	0–127	METRONOME VOLUME

parameter	accepted values	count
00 00 20	00 40 41 42	4
00 10 11	00 40 41 42 03 04 06 10 11 12 13 14 15 16 80	15
00 20 31	00 02 03 04	4

7.6 Names

Every name in the tables is the one Technics’ own parameter tables print for that address. None is inferred. Two of them are corroborated a second way, from a dump taken off a real machine: every part’s PANPOT reads 64, the value the instrument displays as CTR, and the 32 parts’ BASIC CHANNEL bytes read an unbroken ramp from 0 to 31.

The 08 block is marked receivable but not requestable: the instrument accepts a write to it and will not report its contents. Its four addresses are commands rather than settings — they initialise a section, switch play mode, or ask for a store.

7.7 Whole-category request

DRQ also requests an entire bulk-dump category, as described in chapter 5. There ADR names one of the lower five rows of the map above.

7.8 The sound areas

ADR values beginning 10 and 18 reach NORMAL SOUND and DRUM SOUND, the individual sound being edited rather than any stored one. This edition does not enumerate their parameters.

Chapter 8

The sound areas

NORMAL SOUND and DRUM SOUND are the two areas of the address map that hold an instrument sound rather than a setting. They are reached the same way as any other parameter, with ITR to write and DRQ to request.

ADR	address	area
10 00 00	040000	NORMAL SOUND, individual
18 00 00	060000	DRUM SOUND, individual

This is the sound being edited, not a stored one

These addresses reach the instrument's working copy — the sound currently loaded for editing. They are not a way to read or write one of the 32 stored sounds. For those, use the SOUND bulk-dump category, which carries the whole sound memory at 20 00 00.

8.1 How a normal sound is laid out

The NORMAL SOUND area is **713 bytes**. It is one GENERAL DATA block, then four identical TONE DATA blocks, then four identical MODELING DATA blocks — the instrument's four tones and the four driver and resonator models that go with them.

block	size	repeats	parameter numbers
GENERAL DATA	217	1	000–0D8
TONE DATA	81	4	0D9, 12A, 17B, 1CC
MODELING DATA	43	4	21D, 248, 273, 29E
total	713		000–2C8

A parameter number is an offset from the start of the area, so the address of parameter p of tone n (counting tones from 1) is

$$040000 + 0D9 + 81 \times (n - 1) + p$$

expressed as ADR in the usual three seven-bit bytes, and the same with 21D and 43 for a modeling block. The tables below give one instance of each repeating block; the other three are identical at the offsets above.

Two independent sources agree on the size

The block sizes and repeat counts here come from Technics' published parameter tables. The figure of 713 bytes for the whole area comes from the program, decoded separately, and the guide never states it. The transcribed blocks tile 000 to 2C8 with no gap and no overlap and come to exactly that total, which is what makes the layout above trustworthy rather than merely plausible.

8.2 How a drum sound is laid out

The DRUM SOUND area is **19 608 bytes** and is arranged quite differently: a kit, then a table choosing a sound for each of the 128 note numbers, then 128 note records — one per key.

block	size	repeats	parameter numbers
KIT DATA	152	1	000–097
SOUND SELECT	256	1	098–197, two bytes per note
NOTE DATA	150	128	198 + 150 × note
total	19 608		000–4C97

Each note record is itself three kinds of block, and a drum note has *two* tones and two modeling blocks where a normal sound has four:

block	size	repeats	offset within the note
NOTE GENERAL DATA	18	1	000
NOTE TONE DATA	23	2	012, 029
NOTE MODELING DATA	43	2	040, 06B
total	150		

The modeling block is the same 43-byte layout a normal sound uses. The tone block is not: a drum tone has no pitch or filter envelope of its own and no layer ranges, which is what takes it from 81 bytes down to 23.

The note positions are checked against the guide's own table

The guide prints the starting number of every one of the 128 notes. The layout above implies them instead — 198 plus 150 per note — and the two agree wherever they were compared, including at the last row, NOTE:7F, which both put at 4C02. With the area also totalling the 19 608 bytes the program gives, that is three independent routes to the same arithmetic.

8.3 What is transcribed here, and how far it is checked

The tables give each parameter's number, its bit field within that byte, its name and its range. They were read from scans of the published tables, and the numbering is checked by the tiling described above. **The names and bit fields are not checked by anything**; a misreading of one would survive.

A *range* is checked wherever the guide gives it twice, once in hex and once in decimal, which it does on 12 rows. Eleven agree. The twelfth is a misprint in the guide, and is the one place in these tables where the printed page and what is set here differ:

Two misprints in the guide's drum tables

DRUM SOUND / NOTE TONE DATA / PITCH KEY SHIFT, 1AE, is printed with the span E8~18 beside the note -50~+50. E8-18 read as a signed byte is -24 to +24, and the identical row in NORMAL SOUND TONE DATA, 0DD, carries the same span beside -24~+24. The hex is right; the table above prints -24 to +24. The 50 appears to have come from the neighbouring signed rows on the same page, every one of which is CE~32.

On the same page the 2ND column of NOTE TONE DATA is misnumbered. A 23-byte block starting at 012 puts its second instance at 029, and the column's tail (030 onward) agrees — but its first seven entries read 024 to 02A, five too low, and the column jumps from 02A to 030. The numbering above is derived from the stride, so it is unaffected; a reader taking the second tone's addresses off the printed column would write seven parameters to the wrong bytes.

The enumerations these tables point at — the control functions a controller can be assigned to, the filter VALUE bytes for each filter mode, and the INTERACTION CONNECT table — are in chapter 10.

8.4 Normal Sound parameters

GENERAL DATA

no.	bits	parameter	values
000-00F	6-0	NAME (16 characters)	20-7F — space to }
010	7-6	ATTRIBUTE FLAG: PARAMETER MODE	00-00 — 0: NORMAL MODE, fixed
011	7-0	TONE ON/OFF	two bits per tone, 4th 3rd 2nd 1st
012	7-0	KEY ON MODE	per tone: 0 KEY ON, 1 KEY OFF, 2 LEGATO, 3 NON LEGATO
013	7-0	KEY SCALING	00-42 — 0 OFF, 1-22 octave, 64-66 all key
014-016		CONTROLLERS: PITCH BEND	on/off and phase, function select, depth
017-019		MODULATION WHEEL 1 SELECT1	same three bytes
01A-01C		MODULATION WHEEL 1 SELECT2	
01D-01F		MODULATION WHEEL 2 SELECT1	
020-022		MODULATION WHEEL 2 SELECT2	
023	6-0	REALTIME CREATOR BOX SELECT	bit per box, BOX1-BOX6
024	6-0	REALTIME CONTROLLER BOX SELECT	bit per box, BOX1-BOX6
025-027		CONTROL PEDAL SELECT1	
028-02A		CONTROL PEDAL SELECT2	
02B-02D		AFTER TOUCH SELECT1	
02E-030		AFTER TOUCH SELECT2	

no.	bits	parameter	values
031-054		BOX SELECT: CONTROLLER BOX1 X through BOX6 Y	twelve controllers, three bytes each, X then Y
055	3-0	PITCH OCTAVE SHIFT	06-0A — 6 is -2 oct, 8 is 0, A is +2 oct
056		(RESERVE)	
057-066		PITCH LFO BOX1-BOX4	four bytes per box: depth, speed, key sync and touch depth, wave and delay
067-076		LEVEL LFO BOX1-BOX4	same four bytes per box
077-086		FILTER LFO BOX1-BOX4	same four bytes per box
087	3-0, 7-4	MIXER OUTPUT SELECT: MAIN, SUB	00-05 — 0 no output, 1 MAIN, 2-4 SUB1-3, 5 EFFECT2
088	6-0	EFFECT SEND VOLUME: EFFECT1	00-7F
089	6-0	EFFECT SEND VOLUME: REVERB	00-7F
08A	3-0, 7-4	EFFECT OUTPUT SELECT: EFFECT1, EFFECT2	01-04 — 1 MAIN, 2-4 SUB1-3
08B	0	DSP EFFECT COMMON ALGORITHM	00-01 — 0 SERIAL, 1 PARALLEL
08C		EFFECT1 TYPE	see the guide's DSP EFFECT pages
08D-0A0		EFFECT1 PARAMETER VALUE1-VALUE20	meaning depends on the type
0A1		EFFECT1 ANOTHER EFF SEND	
0A2		EFFECT1 DYNAMIC CONTROL	
0A3		EFFECT2 TYPE	
0A4-0B7		EFFECT2 PARAMETER VALUE1-VALUE20	
0B8		EFFECT2 ANOTHER EFF SEND	
0B9		EFFECT2 DYNAMIC CONTROL	
0BA		REVERB TYPE	
0BB-0CE		REVERB PARAMETER VALUE1-VALUE20	
0CF		REVERB DYNAMIC CONTROL	reverb has no ANOTHER EFF SEND
0D0	7, 6, 3-0	DIGITAL EFFECT MODE: ON/OFF, PAN, EFFECT TYPE	00-0B — 0-1 CELESTE, 2-3 CHORUS, 4-5 ENSEMBLE, 6-7 TREMOLO, 8-9 DELAY, 10-11 SOLO EFFECT
0D1-0D8		DIGITAL EFFECT VALUE1-VALUE8	

TONE DATA

no.	bits	parameter	values
0D9	5-0	ATTRIBUTE DELAY	00-32 — 0 delay off
0DA	7-0	ATTRIBUTE PANNING	00-80 — 0-127 fixed pan, 128 random pan
0DB	6-0	TONE SELECT: TTN SELECT	00-7F
0DC	7-6, 5-4, 3-0	TONE KIND, TONE LOCATION, BANK SELECT	kind 0 NORMAL 1 DRUM 2 ATTACK; location 0 internal ROM 1 internal RAM 2 external ROM
0DD	7-0	PITCH KEY SHIFT	E8-18 — -24 to +24
0DE	7-0	PITCH DETUNE	80-7F — -128 to +127
0DF	7-6, 5, 4, 2-0	BOX CONTROL LFO-FM1: BOX SELECT, ON/OFF, PHASE, SCALE SELECT	scale 0 1/1 through 6 1/64, 7 FIX
0E0-0E9		PITCH ENVELOPE SHAPE: total depth, start pitch, attack time, peak level, decay1 time, sustain1 level, decay2 time, sustain2 level, release time, stop pitch	one byte each, in that order
0EA	7-0	PITCH ENV TOUCH FOLLOW ADR TIME	CE-32 — -50 to +50
0EB	7-0	PITCH ENV TOUCH FOLLOW DEPTH	CE-32
0EC	6-0	PITCH ENV KEY FOLLOW CENTER KEY	00-7F
0ED	7-0	PITCH ENV KEY FOLLOW ATTACK SLOPE	CE-32
0EE	7-0	PITCH ENV KEY FOLLOW DECAY SLOPE	CE-32
0EF	7-0	PITCH ENV KEY FOLLOW RELEASE SLOPE	CE-32
0F0	6-0	LEVEL VOLUME	00-7F — one step is 0.375 dB
0F1	7-0	LEVEL TOUCH DEPTH	CE-32
0F2	7-5	LEVEL TOUCH CURVE	00-06 — 0 is -3 curve through 6 is +3 curve
0F3	6-0	LEVEL KEY FOLLOW CENTER KEY	00-7F
0F4	6-0	LEVEL KEY FOLLOW LOW KEY LIMIT	00-7F
0F5	6-0	LEVEL KEY FOLLOW HIGH KEY LIMIT	00-7F
0F6	7-0	LEVEL KEY FOLLOW SLOPE	CE-32
0F7-0FA	6-0	LAYER KEY RANGE: low key select, low fade, high key select, high fade	00-7F
0FB-0FE	6-0	LAYER VELOCITY RANGE: low vel select, low fade, high vel select, high fade	00-7F
0FF	7-6, 5, 4	BOX CONTROL LFO-AM: BOX SELECT, ON/OFF, PHASE	
100-106		LEVEL ENVELOPE SHAPE: attack time, peak level, decay1 time, sustain1 level, decay2 time, sustain2 level, release time	

no.	bits	parameter	values
107	7-0	LEVEL ENV TOUCH FOLLOW ATTACK	CE-32
108	7-0	LEVEL ENV TOUCH FOLLOW DECAY	CE-32
109-10B	6-0	LEVEL ENV KEY FOLLOW: center key, low key limit, high key limit	00-7F
10C-10E	7-0	LEVEL ENV KEY FOLLOW: attack slope, decay slope, release slope	CE-32
10F	2-0, 7-5	FILTER MODE, TOUCH CURVE	00-06 — 0 THROUGH, 1 LPF12+EQ, 2 HPF12+EQ, 3 LPF24, 4 HPF24, 5 BPF
110	5-0	FILTER TOUCH DEPTH	00-32
111	7-6, 5, 4	FILTER LFO-FCM: BOX SELECT, ON/OFF, PHASE	
112-114	6-0	FILTER KEY FOLLOW: center key, low key limit, high key limit	00-7F
115	7-0	FILTER KEY FOLLOW SLOPE	CE-32
116-11F		FILTER ENVELOPE SHAPE: cutoff adjust, start point, attack time, peak level, decay1 time, sustain1 level, decay2 time, sustain2 level, release time, stop point	
120	7-0	FILTER ENV TOUCH FOLLOW ADR TIME	CE-32
121	7-0	FILTER ENV TOUCH FOLLOW DEPTH	CE-32
122	6-0	FILTER ENV KEY FOLLOW CENTER KEY	00-7F
123-125	7-0	FILTER ENV KEY FOLLOW: attack slope, decay slope, release slope	CE-32
126-129		FILTER PARAMETER VALUE1-VALUE4	meaning depends on the filter mode

MODELING DATA

no.	bits	parameter	values
21D-21F	6-0	DRIVER THRESHOLD POINT: between A and B, B and C, C and D	00-7F — provided that $A < B < C < D$
220	6-0	DRIVER A TTN SELECT	00-7F
221	7-6, 5-4, 3-0	DRIVER A KIND, LOCATION, BANK SELECT	kind 0 NORMAL 1 DRUM 2 ATTACK
222-223		DRIVER B: TTN select; kind, location, bank select	
224-225		DRIVER C: TTN select; kind, location, bank select	
226-227		DRIVER D: TTN select; kind, location, bank select	
228	7-6, 5-0	RESONATOR GENERAL: INTERACTION CONNECT, RESONATOR TYPE	00-3F — type 0 ORIGINAL, 1-63 preset

no.	bits	parameter	values
229		(RESERVE)	
22A	7-0	POSITION PARAMETER POSITION	00-FA — 0.2 per cent per step, 0 to 50 per cent
22B	7, 6-0	FORMANT FLAG, DEPTH	00-7F — 0 FIXED, 1 MOVE
22C		(RESERVE)	
22D	7-0	POSITION TOUCH	CE-32 — -50 to +50
22E	5-0	POSITION MOVEMENT WIDTH	00-32
22F	5-0, 7	POSITION MOVEMENT SPEED, SAMPLE/HOLD	00-32
230	6-0	INTERACTION GAIN	00-7F
231		(RESERVE)	
232	7, 6-0	MAIN RESONATOR MODE, FITTING	00-7F — 0 positive feedback, 1 negative feedback
233	7, 6-0	MAIN RESONATOR SCALE, MUTING	00-7F
234	7-0	MAIN TOUCH FOLLOW TOUCH FITTING	CE-32
235	7-0	MAIN TOUCH FOLLOW TOUCH MUTING	CE-32
236-238	6-0	MAIN KEY FOLLOW: center key, low key limit, high key limit	00-7F
239	7-0	MAIN KEY FOLLOW SLOPE	CE-32
23A	7-0	MAIN PITCH KEY SHIFT	C4-3C — -60 to +60
23B	7-0	MAIN PITCH DETUNE	80-7F — -128 to +127
23C	7, 6-0	SUB RESONATOR MODE, FITTING	00-7F
23D	7, 6-0	SUB RESONATOR SCALE, MUTING	00-7F
23E	7-0	SUB GAIN	9C-64
23F	7-0	SUB TOUCH FOLLOW TOUCH FITTING	CE-32
240	7-0	SUB TOUCH FOLLOW TOUCH MUTING	CE-32
241	7-0	SUB TOUCH FOLLOW TOUCH SUB GAIN	CE-32
242-244	6-0	SUB KEY FOLLOW: center key, low key limit, high key limit	00-7F
245	7-0	SUB KEY FOLLOW SLOPE	CE-32
246	7-0	SUB PITCH KEY SHIFT	C4-3C
247	7-0	SUB PITCH DETUNE	80-7F

8.5 Drum Sound parameters

KIT DATA

no.	bits	parameter	values
000-00F	6-0	NAME (16 characters)	20-7F — space to }
010	7-6	ATTRIBUTE FLAG: PARAMETER MODE	00-00 — 2: DRUM MODE, fixed

no.	bits	parameter	values
011-013		CONTROLLERS: PITCH BEND	on/off and phase, function select, depth
014-016		MODULATION WHEEL 1 SELECT1	on/off and phase, function select, depth
017-019		MODULATION WHEEL 1 SELECT2	on/off and phase, function select, depth
01A-01C		MODULATION WHEEL 2 SELECT1	on/off and phase, function select, depth
01D-01F		MODULATION WHEEL 2 SELECT2	on/off and phase, function select, depth
020	6-0	REALTIME CREATOR BOX SELECT	bit per box, BOX1-BOX6
021	6-0	REALTIME CONTROLLER BOX SELECT	bit per box, BOX1-BOX6
022-024		CONTROL PEDAL SELECT1	on/off and phase, function select, depth
025-027		CONTROL PEDAL SELECT2	on/off and phase, function select, depth
028-02A		AFTER TOUCH SELECT1	on/off and phase, function select, depth
02B-02D		AFTER TOUCH SELECT2	on/off and phase, function select, depth
02E-051		BOX SELECT: CONTROLLER BOX1 X through BOX6 Y	twelve controllers, three bytes each, X then Y
052	3-0, 7-4	MIXER EFFECT OUTPUT SELECT: EFFECT1, EFFECT2	01-04 — 1 MAIN, 2-4 SUB1-3
053	0	DSP EFFECT COMMON ALGORITHM	00-01 — 0 SERIAL, 1 PARALLEL
054		EFFECT1 TYPE	see the effect tables
055-068		EFFECT1 PARAMETER VALUE1-VALUE20	meaning depends on the type
069		EFFECT1 ANOTHER EFF SEND	
06A		EFFECT1 DYNAMIC CONTROL	
06B		EFFECT2 TYPE	
06C-07F		EFFECT2 PARAMETER VALUE1-VALUE20	
080		EFFECT2 ANOTHER EFF SEND	
081		EFFECT2 DYNAMIC CONTROL	
082		REVERB TYPE	
083-096		REVERB PARAMETER VALUE1-VALUE20	
097		REVERB DYNAMIC CONTROL	reverb has no ANOTHER EFF SEND

SOUND SELECT

no.	bits	parameter	values
098-197		SOUND SELECT, NOTE:00 to NOTE:7F	128 notes, two bytes each

NOTE GENERAL DATA

no.	bits	parameter	values
198-1A4	6-0	NAME (13 characters)	20-7F
1A5	5, 3-0	ATTRIBUTE FLAG: KEY OFF MODE, TONE ON/OFF	00-01 — two bits per tone, 2nd then 1st
1A6	7, 6-0	SAME GROUP DUMP: DUMP FLAG, GROUP NUMBER	00-7F — flag 1 dumps the same group's sound
1A7	3-0, 7-4	MIXER OUTPUT SELECT: MAIN, SUB	00-05 — 0 no output, 1 MAIN, 2-4 SUB1-3, 5 EFFECT2
1A8	6-0	EFFECT SEND VOLUME: EFFECT1	00-7F
1A9	6-0	EFFECT SEND VOLUME: REVERB	00-7F

NOTE TONE DATA

no.	bits	parameter	values
1AA	5-0	ATTRIBUTE DELAY	00-32 — 0 delay off
1AB	7-0	ATTRIBUTE PANNING	00-80 — 0-127 fixed pan, 128 random pan
1AC	6-0	TONE SELECT: TTN SELECT	00-7F
1AD	7-6, 5-4, 3-0	TONE KIND, TONE LOCATION, BANK SELECT	kind 0 NORMAL 1 DRUM 2 ATTACK
1AE	7-0	PITCH KEY SHIFT	E8-18 — -24 to +24
1AF	7-0	PITCH DETUNE	80-7F — -128 to +127
1B0	6-0	LEVEL VOLUME	00-7F — one step is 0.375 dB
1B1	7-0	LEVEL TOUCH DEPTH	CE-32 — -50 to +50
1B2	7-5	LEVEL TOUCH CURVE	00-06 — 0 is -3 curve through 6 is +3 curve
1B3-1B8	6-0	LEVEL ENVELOPE SHAPE: attack time, decay1 time, sustain1 level, decay2 time, sustain2 level, release time	00-64 — one byte each, in that order
1B9	7-0	LEVEL ENV TOUCH FOLLOW ATTACK	CE-32
1BA	7-0	LEVEL ENV TOUCH FOLLOW DECAY	CE-32
1BB	2-0, 7-5	FILTER MODE, TOUCH CURVE	00-06 — 0 THROUGH, 1 LPF12+EQ, 2 HPF12+EQ, 3 LPF24, 4 HPF24, 5 BPF
1BC	5-0	FILTER TOUCH DEPTH	00-32
1BD-1C0		FILTER PARAMETER VALUE1-VALUE4	meaning depends on the filter mode

NOTE MODELING DATA

no.	bits	parameter	values
1D8-1DA	6-0	DRIVER THRESHOLD POINT: between A and B, B and C, C and D	00-7F — provided that $A < B < C < D$
1DB	6-0	DRIVER A TTN SELECT	00-7F
1DC	7-6, 5-4, 3-0	DRIVER A KIND, LOCATION, BANK SELECT	kind 0 NORMAL 1 DRUM 2 ATTACK
1DD-1DE		DRIVER B: TTN select; kind, location, bank select	
1DF-1E0		DRIVER C: TTN select; kind, location, bank select	
1E1-1E2		DRIVER D: TTN select; kind, location, bank select	
1E3	7-6, 5-0	RESONATOR GENERAL: INTERACTION CONNECT, RESONATOR TYPE	00-3F — type 0 ORIGINAL, 1-63 preset
1E4		(RESERVE)	
1E5	7-0	POSITION PARAMETER POSITION	00-FA — 0.2 per cent per step, 0 to 50 per cent
1E6	7, 6-0	FORMANT FLAG, DEPTH	00-7F — 0 FIXED, 1 MOVE
1E7		(RESERVE)	
1E8	7-0	POSITION TOUCH	CE-32 — -50 to +50
1E9	5-0	POSITION MOVEMENT WIDTH	00-32
1EA	5-0, 7	POSITION MOVEMENT SPEED, SAMPLE/HOLD	00-32
1EB	6-0	INTERACTION GAIN	00-7F
1EC		(RESERVE)	
1ED	7, 6-0	MAIN RESONATOR MODE, FITTING	00-7F — 0 positive feedback, 1 negative feedback
1EE	7, 6-0	MAIN RESONATOR SCALE, MUTING	00-7F
1EF	7-0	MAIN TOUCH FOLLOW TOUCH FITTING	CE-32
1F0	7-0	MAIN TOUCH FOLLOW TOUCH MUTING	CE-32
1F1-1F3	6-0	MAIN KEY FOLLOW: center key, low key limit, high key limit	00-7F
1F4	7-0	MAIN KEY FOLLOW SLOPE	CE-32
1F5	7-0	MAIN PITCH KEY SHIFT	C4-3C — -60 to +60
1F6	7-0	MAIN PITCH DETUNE	80-7F — -128 to +127
1F7	7, 6-0	SUB RESONATOR MODE, FITTING	00-7F
1F8	7, 6-0	SUB RESONATOR SCALE, MUTING	00-7F
1F9	7-0	SUB GAIN	9C-64
1FA	7-0	SUB TOUCH FOLLOW TOUCH FITTING	CE-32
1FB	7-0	SUB TOUCH FOLLOW TOUCH MUTING	CE-32
1FC	7-0	SUB TOUCH FOLLOW TOUCH SUB GAIN	CE-32
1FD-1FF	6-0	SUB KEY FOLLOW: center key, low key limit, high key limit	00-7F
200	7-0	SUB KEY FOLLOW SLOPE	CE-32

no.	bits	parameter	values
201	7-0	SUB PITCH KEY SHIFT	C4-3C
202	7-0	SUB PITCH DETUNE	80-7F

Chapter 9

Effects

A sound carries three effect blocks — EFFECT1, EFFECT2 and REVERB — and each is a TYPE byte followed by twenty VALUE bytes, an ANOTHER EFF SEND and a DYNAMIC CONTROL. The TYPE byte selects one of 56 effects, and what the twenty VALUE bytes mean depends entirely on which.

9.1 What the type byte selects

The numbers are not contiguous: they are grouped, with modulation effects below 10, reverbs at 10–1B, distortion and dynamics at 20–27, a second modulation group at 30–38, two-effect combinations at 40–4B and three-effect combinations at 60–63.

Twelve of the reverbs are marked (REV ONLY) in the published guide: they are intended for the REVERB block rather than EFFECT1 or EFFECT2.

Two sources agree on which effects exist

The table below was transcribed from the guide. Independently, this project disassembled the effects DSP and produced a manifest of every distinct effect program with the number that selects it. The two sets of numbers are equal — 56 effects, no more on either side — which is what makes the numbering here trustworthy.

They disagree only in spelling, because the ROM carries the short names the instrument displays and the guide prints long ones. The table gives the guide's.

9.2 What the value bytes mean

Each parameter below is printed with the VALUE byte it occupies, in bold. Those numbers are the guide's own: every DSP EFFECT page carries a MIDI VALUE column giving each row's byte outright. An effect that uses fewer than twenty leaves the rest unused.

A parameter is not always one byte, and a byte is not always one parameter

It is tempting to read the parameter list as the byte order — first name VALUE1, second VALUE2. That is true of 34 of the 56 effects and wrong for the other 22, so a librarian built on it addresses those 22 incorrectly.

Two things break it. A parameter with a wide range spans *two* bytes: SINGLE DELAY reaches 350 ms and its DELAY L is bytes 1–2, which pushes everything after it up by one. And several parameters can *share* a pair: each band of PARAMETRIC EQ names an FC, a Q and a G, and all three are packed into the same two bytes by the guide's table 17. PARAMETRIC EQ has 19 parameters and 13 VALUE bytes.

Where a span like **2–3** appears against more than one parameter, those parameters share those bytes; the packing tables at the end of this chapter give each field's width and position.

The combination effects concatenate their components: the first block's parameters, then a dry/wet for the second block, then the second block's. A three-effect combination continues the same way.

The names are transcribed, and not checked by anything

The effect *numbers* are confirmed twice over, and the byte numbers check themselves: each effect's must be exactly $1 \dots N$, used once each in ascending order, which a misread digit almost always breaks. The *names* have no such check — they were read from scans, nothing independent states them, and a misreading would survive. Treat them as a guide to what the bytes are for rather than as a specification.

The block diagram the guide prints for each effect is not reproduced either.

no.	n	effect	parameters: VALUE byte, then the conversion table
00	0	NO OPERATION	—
01	5	CHORUS	WET 1, DEPTH 2, LFO SPEED 3*3, LFO WAVEFORM 4*4, VOLUME 5
02	7	MODULATED CHORUS	WET 1, DEPTH 2, SLOW LFO SPEED 3*3, FAST LFO SPEED 4*3, FAST LFO BALANCE 5, LFO WAVEFORM 6*4, VOLUME 7
03	9	ENHANCER	WET 1, MANUAL 2, LOW MIX 3, HIGH MIX 4, DELAY TIME L 5–6, DELAY TIME R 7–8, VOLUME 9
04	8	FLANGER	WET 1, DEPTH 2, LFO SPEED 3*3, RESONANCE 4, MANUAL 5, PHASE 6, LFO WAVEFORM 7*4, VOLUME 8
05	8	PHASER	WET 1, DEPTH 2, LFO SPEED 3*3, RESONANCE 4, MANUAL 5, PHASE 6, LFO WAVEFORM 7*4, VOLUME 8
06	5	ENSEMBLE	WET 1, DEPTH 2, LFO SPEED 3*3, LFO WAVEFORM 4*4, VOLUME 5
08	6	GATED REVERB	WET 1, GATE TIME 2*5, HIGH DUMP GAIN 3*2, THRESHOLD 4, MASK TIME 5*5, VOLUME 6
09	9	SINGLE DELAY	WET 1, DELAY L 2–3, DELAY R 4–5, FEEDBACK L 6, FEEDBACK R 7, HIGH DUMP GAIN 8*2, VOLUME 9

no.	n	effect	parameters: VALUE byte, then the conversion table
0A	16	MULTI TAP DELAY	WET 1 , DELAY 1 2–3 , DELAY 2 4–5 , DELAY 3 6–7 , DELAY 4 8–9 , PAN 1 10 , PAN 2 11 , PAN 3 12 , PAN 4 13 , FEED BACK 14 , HIGH DUMP GAIN 15 ^{*2} , VOLUME 16
0B	10	MANUAL DELAY	WET 1 , MODULATION DEPTH 2 , DELAY L 3–4 , DELAY R 5–6 , FEEDBACK L 7 , FEEDBACK R 8 , HIGH DUMP GAIN 9 ^{*2} , VOLUME 10
10	5	ROOM REVERB 1 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
11	5	ROOM REVERB 2 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
12	5	PLATE REVERB 1 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
13	5	PLATE REVERB 2 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
14	5	CONCERT REVERB 1 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
15	5	CONCERT REVERB 2 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
16	5	DARK REVERB 1 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
17	5	DARK REVERB 2 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
18	5	BRIGHT REVERB 1 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
19	5	BRIGHT REVERB 2 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
1A	5	WAVE REVERB 1 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
1B	5	WAVE REVERB 2 (REV only)	REVERB TIME 1 ^{*1} , PRE DELAY 2 , HIGH DUMP GAIN 3 ^{*2} , EARLY REFL LEVEL 4 , VOLUME 5
20	4	DISTORTION	WET 1 , DRIVE 2 , ADJUST 3 , VOLUME 4
21	4	OVERDRIVE	WET 1 , DRIVE 2 , ADJUST 3 , VOLUME 4
22	4	FUZZ	WET 1 , DRIVE 2 , ADJUST 3 , VOLUME 4
23	7	EXCITER	WET 1 , DRIVE 2 , ADJUST 3 , EMPHASIS Fc 4–5 ^{*6,18} , EMPHASIS GAIN 6 , VOLUME 7
24	6	COMPRESSOR	WET 1 , THRESHOLD 2 , RATIO 3 , ATTACK SENSITIVITY 4 ^{*9} , RELEASE SENSITIVITY 5 ^{*9} , VOLUME 6
25	5	SLOW ATTACKER	WET 1 , THRESHOLD 2 , ATTACK RATE 3 ^{*10} , RELEASE RATE 4 ^{*10} , VOLUME 5
26	1	NOISE GENERATOR	VOLUME 1

no.	n	effect	parameters: VALUE byte, then the conversion table
27	13	PARAMETRIC EQ	BAND EMPHASIS 1 Fc 1–2* ⁶ , BAND EMPHASIS 1 Q 1–2* ⁷ , BAND EMPHASIS 1 G 1–2* ⁸ , BAND EMPHASIS 2 Fc 3–4* ⁶ , BAND EMPHASIS 2 Q 3–4* ⁷ , BAND EMPHASIS 2 G 3–4* ⁸ , BAND EMPHASIS 3 Fc 5–6* ⁶ , BAND EMPHASIS 3 Q 5–6* ⁷ , BAND EMPHASIS 3 G 5–6* ⁸ , BAND EMPHASIS 4 Fc 7–8* ⁶ , BAND EMPHASIS 4 Q 7–8* ⁷ , BAND EMPHASIS 4 G 7–8* ⁸ , BAND EMPHASIS 5 Fc 9–10* ⁶ , BAND EMPHASIS 5 Q 9–10* ⁷ , BAND EMPHASIS 5 G 9–10* ⁸ , BAND EMPHASIS 6 Fc 11–12* ⁶ , BAND EMPHASIS 6 Q 11–12* ⁷ , BAND EMPHASIS 6 G 11–12* ⁸ , VOLUME 13
30	6	AUTO PAN	WET 1, DEPTH 2, LFO SPEED 3* ³ , PHASE 4, LFO WAVEFORM 5* ⁴ , VOLUME 6
31	6	PITCH SHIFTER	WET 1, PITCH L 2* ¹⁴ , PITCH R 3* ¹⁴ , PRE DELAY 4, FEEDBACK 5, VOLUME 6
32	6	VIBRATO	WET 1, DEPTH 2, LFO SPEED 3* ³ , PHASE 4, LFO WAVEFORM 5* ⁴ , VOLUME 6
33	6	PEDAL WAH	WET 1, RESONANCE 2* ¹⁵ , MANUAL 3, SWEEP RANGE 4, WAH CENTER Fc 5, VOLUME 6
34	5	AUTO WAH	WET 1, RESONANCE 2* ¹⁵ , MANUAL 3, SWEEP RANGE 4, VOLUME 5
35	15	ROTARY SPEAKER	WET 1, DRIVE 2, VOLUME ADJUST 3, TREBLE DEPTH 4, TREBLE FAST 5* ¹¹ , TREBLE SLOW 6* ¹¹ , TREBLE WIND UP 7* ¹² , TREBLE WIND DOWN 8* ¹² , BASS DEPTH 9, BASS FAST 10* ¹¹ , BASS SLOW 11* ¹¹ , BASS WIND UP 12* ¹² , BASS WIND DOWN 13* ¹² , VOLUME 14, SLOW/FAST 15* ¹⁶
36	5	RING MODULATOR	WET 1, OSC SPEED 2* ¹³ , PHASE 3, OSC WAVEFORM 4* ⁴ , VOLUME 5
37	8	HAAS EFFECT	WET 1, DELAY TIME L 2–3, DELAY TIME R 4–5, BALANCE L 6, BALANCE R 7, VOLUME 8
38	8	MIX UP	WET 1, DEPTH 2, SLOW LFO SPEED 3* ³ , FAST LFO SPEED L 4* ³ , FAST LFO SPEED R 5* ³ , PHASE 6, LFO WAVEFORM 7* ⁴ , VOLUME 8
40	12	SINGLE DELAY + CHORUS	DELAY WET 1, DELAY L 2–3, DELAY R 4–5, FEEDBACK L 6, FEEDBACK R 7, CHORUS DRY/WET 8, DEPTH 9, LFO SPEED 10* ³ , LFO WAVEFORM 11* ⁴ , VOLUME 12
41	11	SINGLE DELAY + SINGLE DELAY	DELAY 1 WET 1, DELAY L 2, DELAY R 3, FEEDBACK L 4, FEEDBACK R 5, DELAY 2 DRY/WET 6, DELAY L 7, DELAY R 8, FEEDBACK L 9, FEEDBACK R 10, VOLUME 11
42	15	SINGLE DELAY + FLANGER	DELAY WET 1, DELAY L 2–3, DELAY R 4–5, FEEDBACK L 6, FEEDBACK R 7, FLANGER DRY/WET 8, DEPTH 9, LFO SPEED 10* ³ , RESONANCE 11, MANUAL 12, PHASE 13, LFO WAVEFORM 14* ⁴ , VOLUME 15

no.	n	effect	parameters: VALUE byte, then the conversion table
43	13	SINGLE DELAY + VIBRATO	DELAY WET 1, DELAY L 2–3, DELAY R 4–5, FEEDBACK L 6, FEEDBACK R 7, VIBRATO DRY/WET 8, DEPTH 9, LFO SPEED 10* ³ , PHASE 11, LFO WAVEFORM 12* ⁴ , VOLUME 13
44	15	SINGLE DELAY + PHASER	DELAY WET 1, DELAY L 2–3, DELAY R 4–5, FEEDBACK L 6, FEEDBACK R 7, PHASER DRY/WET 8, DEPTH 9, LFO SPEED 10* ³ , RESONANCE 11, MANUAL 12, PHASE 13, LFO WAVEFORM 14* ⁴ , VOLUME 15
45	13	PEDAL WAH + SINGLE DELAY	WAH WET 1, RESONANCE 2* ¹⁵ , MANUAL 3, SWEEP RANGE 4, WAH CENTER Fc 5, DELAY DRY/WET 6, DELAY L 7–8, DELAY R 9–10, FEEDBACK L 11, FEEDBACK R 12, VOLUME 13
46	12	AUTO WAH + SINGLE DELAY	WAH WET 1, RESONANCE 2* ¹⁵ , MANUAL 3, SWEEP RANGE 4, DELAY DRY/WET 5, DELAY L 6–7, DELAY R 8–9, FEEDBACK L 10, FEEDBACK R 11, VOLUME 12
47	9	PEQ + CHORUS	BAND EMPHASIS 1 Fc 1–2* ⁶ , BAND EMPHASIS 1 Q 1–2* ⁷ , BAND EMPHASIS 1 G 1–2* ⁸ , BAND EMPHASIS 2 Fc 3–4* ⁶ , BAND EMPHASIS 2 Q 3–4* ⁷ , BAND EMPHASIS 2 G 3–4* ⁸ , CHORUS DRY/WET 5, DEPTH 6, LFO SPEED 7, LFO WAVEFORM 8* ⁴ , VOLUME 9
48	12	PEQ + SINGLE DELAY	BAND EMPHASIS 1 Fc 1–2* ⁶ , BAND EMPHASIS 1 Q 1–2* ⁷ , BAND EMPHASIS 1 G 1–2* ⁸ , BAND EMPHASIS 2 Fc 3–4* ⁶ , BAND EMPHASIS 2 Q 3–4* ⁷ , BAND EMPHASIS 2 G 3–4* ⁸ , DELAY DRY/WET 5, DELAY L 6–7, DELAY R 8–9, FEEDBACK L 10, FEEDBACK R 11, VOLUME 12
49	12	PEQ + FLANGER	BAND EMPHASIS 1 Fc 1–2* ⁶ , BAND EMPHASIS 1 Q 1–2* ⁷ , BAND EMPHASIS 1 G 1–2* ⁸ , BAND EMPHASIS 2 Fc 3–4* ⁶ , BAND EMPHASIS 2 Q 3–4* ⁷ , BAND EMPHASIS 2 G 3–4* ⁸ , FLANGER DRY/WET 5, DEPTH 6, LFO SPEED 7* ³ , RESONANCE 8, MANUAL 9, PHASE 10, LFO WAVEFORM 11* ⁴ , VOLUME 12
4A	10	PEQ + VIBRATO	BAND EMPHASIS 1 Fc 1–2* ⁶ , BAND EMPHASIS 1 Q 1–2* ⁷ , BAND EMPHASIS 1 G 1–2* ⁸ , BAND EMPHASIS 2 Fc 3–4* ⁶ , BAND EMPHASIS 2 Q 3–4* ⁷ , BAND EMPHASIS 2 G 3–4* ⁸ , VIBRATO DRY/WET 5, DEPTH 6, LFO SPEED 7* ³ , PHASE 8, LFO WAVEFORM 9* ⁴ , VOLUME 10
4B	9	PEQ + COMPRESSOR	BAND EMPHASIS 1 Fc 1–2* ⁶ , BAND EMPHASIS 1 Q 1–2* ⁷ , BAND EMPHASIS 1 G 1–2* ⁸ , BAND EMPHASIS 2 Fc 3–4* ⁶ , BAND EMPHASIS 2 Q 3–4* ⁷ , BAND EMPHASIS 2 G 3–4* ⁸ , THRESHOLD 5, RATIO 6, ATTACK SENSITIVITY 7* ⁹ , RELEASE SENSITIVITY 8* ⁹ , VOLUME 9

no.	n	effect	parameters: VALUE byte, then the conversion table
60	11	PEQ + COMPRESSOR + DISTORTION	BAND EMPHASIS 1 Fc 1–2 ^{*6} , BAND EMPHASIS 1 Q 1–2 ^{*7} , BAND EMPHASIS 1 G 1–2 ^{*8} , BAND EMPHASIS 2 Fc 3–4 ^{*6} , BAND EMPHASIS 2 Q 3–4 ^{*7} , BAND EMPHASIS 2 G 3–4 ^{*8} , THRESHOLD 5, RATIO 6, ATTACK SENSITIVITY 7 ^{*9} , RELEASE SENSITIVITY 8 ^{*9} , DRIVE 9, ADJUST 10, VOLUME 11
61	9	PEQ + COMPRESSOR + OVERDRIVE	BAND EMPHASIS Fc 1–2 ^{*6} , BAND EMPHASIS Q 1–2 ^{*7} , BAND EMPHASIS G 1–2 ^{*8} , THRESHOLD 3, RATIO 4, ATTACK SENSITIVITY 5 ^{*9} , RELEASE SENSITIVITY 6 ^{*9} , DRIVE 7, ADJUST 8, VOLUME 9
62	14	PEQ + DISTORTION + DELAY	BAND EMPHASIS 1 Fc 1–2 ^{*6} , BAND EMPHASIS 1 Q 1–2 ^{*7} , BAND EMPHASIS 1 G 1–2 ^{*8} , BAND EMPHASIS 2 Fc 3–4 ^{*6} , BAND EMPHASIS 2 Q 3–4 ^{*7} , BAND EMPHASIS 2 G 3–4 ^{*8} , DRIVE 5, ADJUST 6, DELAY DRY/WET 7, DELAY L 8–9, DELAY R 10–11, FEEDBACK L 12, FEEDBACK R 13, VOLUME 14
63	14	PEQ + OVERDRIVE + DELAY	BAND EMPHASIS 1 Fc 1–2 ^{*6} , BAND EMPHASIS 1 Q 1–2 ^{*7} , BAND EMPHASIS 1 G 1–2 ^{*8} , BAND EMPHASIS 2 Fc 3–4 ^{*6} , BAND EMPHASIS 2 Q 3–4 ^{*7} , BAND EMPHASIS 2 G 3–4 ^{*8} , DRIVE 5, ADJUST 6, DELAY DRY/WET 7, DELAY L 8–9, DELAY R 10–11, FEEDBACK L 12, FEEDBACK R 13, VOLUME 14

9.3 From a byte to a quantity

A VALUE byte is not a quantity. The tables below turn one into seconds, hertz, decibels or cents. Which table applies is printed in the table above as a superscript against each parameter: HIGH DUMP GAIN 8^{*2} is byte 8, read through table 2. A parameter with no superscript needs no table — the range beside it in the guide is already the quantity, and the byte carries it directly.

Of the 435 parameters, 159 take a table and 276 are direct. The assignment is the guide's own: each DSP EFFECT page carries a MIDI column pair, and while the VALUE column gives the byte number, the DATA column beside it names the table.

One parameter where the guide contradicts itself

Effect 47, PEQ + CHORUS, prints no table against its LFO SPEED and no HZ against its range. The same parameter with the same 0–40.2 range carries table 3 and the unit on all eleven other pages that have it. The table above prints what page 27 prints; a librarian would do better to treat it as table 3 like its siblings.

Each row gives a span of parameter values, the quantity at each end of it, and the step between one parameter and the next.

Three of the guide's tables are not conversions at all but *packings*: they say how several

fields share two bytes. Those are given at the end, with each field's width.

The tables check themselves, twice over

The four numbers in a row are not independent: the first value plus the step times the span must equal the last. All 41 rows satisfy that identity, and a misread endpoint or step would break it.

The packings check themselves differently. A field's width must be wide enough for the range of the table it carries, and each one is exactly wide enough — five bits for a parameter reaching 31, five for one reaching 26, six for one reaching 48.

Table 1: Reverb time (seconds)

parameter	value	step
0–15	0.1–0.4	0.02
16–23	0.45–0.8	0.05
24–55	0.9–4	0.1
56–75	4.2–8	0.2
76–97	9–30	1

Table 2: High dump gain (dB)

parameter	value	step
0–24	-24–0	1

Table 3: LFO speed (Hz)

parameter	value	step
0–50	0–5	0.1
51–75	5.2–10	0.2
76–88	10.8–20.4	0.8
89–99	22.2–40.2	1.8

Table 4: LFO wave form 00 sine, 01 triangle, 02 square.

Table 5: Gate time and mask time, gated reverb (ms)

parameter	value	step
0–20	10–50	2
21–30	55–100	5
31–70	110–500	10
71–80	550–1000	50
81–99	1100–2900	100

Table 6: PEQ Fc (Hz) A third-octave series; the parameter is the index.

Parameter 00–1A selects, in order: 40, 50, 63, 80, 100, 125, 160, 200, 250, 315, 400, 500, 630, 800, 1000, 1250, 1600, 2000, 2500, 3150, 4000, 5000, 6300, 8000, 10000, 12500, 16000.

Table 7: PEQ Q

parameter	value	step
0–9	0.1–1	0.1
10–15	1.5–4	0.5
16–31	5–20	1

Table 8: PEQ gain (dB)

parameter	value	step
0–48	-12–12	0.5

Table 9: Attack and release sensitivity (seconds, 90 per cent swing time)

parameter	value	step
0–89	0.001–0.09	0.001
90–90	0.1–0.1	0.01
91–94	0.2–0.5	0.1

Table 10: Attack for slow attacker (seconds, 90 per cent swing time) The first row is attack, the second release.

parameter	value	step
0–99	0.2–20	0.2
0–99	0.01–1	0.01

Table 11: LFO speed, rotary speaker (Hz)

parameter	value	step
0–20	0–2	0.1
21–50	2.2–8	0.2
51–99	8.55–34.95	0.55

Table 12: LFO acceleration, rotary speaker (seconds, 90 per cent swing time)

parameter	value	step
0–75	1–31	0.4
76–99	32.25–61	1.25

Table 13: LFO speed, ring modulator (Hz)

parameter	value	step
0–10	0–10	1
11–19	20–100	10
20–64	120–1000	20
65–87	1200–10000	400
88–99	10800–19600	800

Table 14: Pitch L and R, pitch shifter (cents)

parameter	value	step
-36—32	-1200—800	100
-31—20	-700—150	50
-19—11	-100—20	10
-10—10	-10—10	1
11—19	20—100	10
20—31	150—700	50
32—36	800—1200	100

Table 15: Resonance for wah 00 wide, 01 middle, 02 narrow.**Table 16: Slow/Fast, rotary speaker** 00 slow, 01 fast.

Packed fields Three of the guide’s tables describe fields sharing two bytes rather than a conversion. Each is listed with its width.

format	field	width
PEQ Q, Fc and G	PEQ Q	5 bits
	PEQ Fc	5 bits — split 3 bits in the first byte, 2 in the second
	PEQ G	6 bits
Exciter emphasis Fc	Emphasis Fc	5 bits — split 3 bits and 2, the same shape as table 17
Pre/post equaliser Fc and G	EQ Fc	5 bits — split 3 bits and 2
	EQ G	6 bits

Chapter 10

Enumerations

Three lists in chapter 8 are referred to rather than spelled out where they are used, because each is shared by many parameters. They are given here.

10.1 Control functions

Every controller in a sound — the pitch bend lever, the two modulation wheels, the control pedal, after touch, and the six controller boxes in each axis — is three bytes: an on/off and phase byte, a FUNCTION SELECT byte, and a depth. The FUNCTION SELECT byte chooses what the controller moves, from this list.

00	OFF	1A	filter LFO3 speed
01	pitch bend	1B	filter LFO4 speed
02	sustain level	1C	fitting*
03	filter cutoff frequency	1D	position
04	pitch LFO1 depth	1E	position depth
05	pitch LFO2 depth	1F	(reserve)
06	pitch LFO3 depth	20	position movement width
07	pitch LFO4 depth	21	position movement speed
08	amp LFO1 depth	22	(reserve)
09	amp LFO2 depth	23	muting*
0A	amp LFO3 depth	24	resonator key shift*
0B	amp LFO4 depth	25	sub gain
0C	filter LFO1 depth	26	delay*
0D	filter LFO2 depth	27	panning*
0E	filter LFO3 depth	28	effect1 send*
0F	filter LFO4 depth	29	reverb send*
10	pitch LFO1 speed	2A	level*
11	pitch LFO2 speed	2B	amp env. attack time*
12	pitch LFO3 speed	2C	amp env. decay time*
13	pitch LFO4 speed	2D	amp env. release time*
14	amp LFO1 speed	2E	filter resonance*
15	amp LFO2 speed	2F	effect1 dynamic control
16	amp LFO3 speed	30	effect2 dynamic control
17	amp LFO4 speed	31	reverb dynamic control
18	filter LFO1 speed	32–7F	(reserve)
19	filter LFO2 speed		

* marked functions cannot be driven by after touch.

A drum kit has its own, much shorter version of this list: most of the numbers above are reserved there, and only OFF, PITCH BEND, SUSTAIN LEVEL, FILTER CUTOFF FREQUENCY, the resonator and position entries and the three dynamic controls remain.

10.2 Filter values

A tone’s filter has four VALUE bytes whose meaning depends on the filter mode selected for that tone.

mode	VALUE1	VALUE2	VALUE3	VALUE4
LPF12	cutoff frequency	resonance	EQ frequency	EQ gain, EQ range
HPF12	cutoff frequency	resonance	EQ frequency	EQ gain, EQ range
LPF24	cutoff frequency	resonance	—	—
HPF24	cutoff frequency	resonance	—	—
BPF	low cutoff frequency	low resonance	high cutoff frequency	high resonance

Cutoff frequencies are 00–7F. Resonance is 00–05: in the 12 dB modes and BPF a step is 3 dB, covering –6 to 9 dB; in the 24 dB modes a step is 6 dB, covering –12 to 18 dB. EQ GAIN is 00–0E, where 00 is silence and 01–0E span –6 to +6 dB, and the top bit of that same byte is EQ RANGE: 0 low, 1 high.

10.3 Interaction connect

Modeling parameter 228 sets how a sound’s four resonators are connected — none of them, the first pair, the second pair, both pairs, or all four together. The guide gives the resulting connection state of each of the four blocks for every setting; it is not reproduced here, and the parameter’s own range, 00–03, is in chapter 8.

Chapter 11

Universal System Exclusive

11.1 Recognised messages

The instrument honours two universal messages, and transmits both.

message	meaning
F0 7E 7F 09 01 F7	General MIDI System On
F0 7E 7F 09 02 F7	General MIDI System Off

Turning General MIDI on restricts the sounds and operations available and may rearrange the percussion map. General MIDI System Off has no effect if General MIDI is already off.

A received message takes effect at once. The same change made at the front panel does not: the panel asks for a confirmation first, and the user can decline it. A controller sending General MIDI System On is therefore doing something the user cannot be offered the chance to refuse, and the mode change it causes is one the instrument warns will make material recorded in the other mode play back wrongly.

Both messages are also written into Standard MIDI Files produced by the instrument, as a five-byte System Exclusive event at delta time zero.

11.2 Messages not recognised

A controller expecting any of the following will receive no response:

message	
F0 7F 7F 04 01 <i>ll mm</i> F7	Master Volume
F0 7E <i>dd</i> 06 01 F7	Identity Request
F0 7E 7F 09 03 F7	General MIDI 2 System On

The instrument does not implement Identity Reply, so it cannot be auto-detected by a controller that relies on it. The entire Universal Real Time identifier 7F is unrecognised.

11.3 Practical consequences

A General MIDI message sent during a bulk dump breaks the dump. While a transfer is open the instrument accepts only messages beginning F0 50. One beginning F0 7E is treated as a reception failure: the transfer is abandoned and ERROR 41 is reported. General MIDI mode is left unchanged.

The instrument announces its own changes. Entering or leaving General MIDI mode makes it transmit the matching message, for a change made at the front panel as well as one made by loading a song. Neither direction is affected by the EXCLUSIVE filter.

The announcement stops for good once you send one

The interlock that stops the instrument echoing a General MIDI message back is armed by the first such message it receives after power-on, and is never disarmed. Once a controller has sent either message, the instrument stops announcing its own General MIDI changes for the rest of the session. A controller that tracks the instrument's mode by listening for these messages will silently stop being told.

General MIDI state travels in a dump. It is carried in the second SYSTEM, PART & MIDI block, so restoring that category restores the General MIDI setting with it. A dump taken in General MIDI mode is not the same file as one taken out of it.

11.4 The identifier is checked once

Once 50 or 7E has been accepted the instrument does not look at it again; the third byte alone selects the message. F0 50 7F 09 01 F7 is therefore acted on as General MIDI System On despite carrying the Matsushita identifier, and a Technics message sent with 7E in the identifier position works equally well. Send the conventional identifier, but expect a receiver to be lenient.

Nothing else about General MIDI mode changes the System Exclusive implementation: every family is accepted exactly as before, with the same encoding, checksum, handshake and error reporting.

Chapter 12

Settings that affect System Exclusive

12.1 The EXCLUSIVE filter

The MIDI menu's INPUT & OUTPUT FILTER page has eight rows, each a plain ON/OFF switch for one kind of MIDI message: PROGRAM CHANGE, BANK SELECT, PITCH BEND, CONTROL CHANGE, RESET ALL CNT., CHANNEL PRESSURE, SONG SELECT and, last, EXCLUSIVE. There is one such switch and it governs both directions: the same setting is consulted when the instrument sends and when it receives. There is no separate input filter, no per-socket filter and no third state, and the page is the same on both models.

EXCLUSIVE governs the tempo message and nothing else

Despite its name, this switch does not turn System Exclusive on and off. Its entire effect on the messages in this reference is the family 25 tempo message: with EXCLUSIVE set to OFF the instrument neither sends one nor acts on one received, silently in both directions.

A bulk dump transfers normally with EXCLUSIVE set to OFF. Software that cannot complete a transfer should not look here for the cause.

This is worth knowing before telling a user to change the setting: the illustration of that page in Technics' own guide shows EXCLUSIVE switched OFF, so an instrument presenting that page as pictured is already able to send and receive every message in this reference.

Everything below is unaffected by the setting:

function	affected by EXCLUSIVE?
bulk dump, sent or received	no
dump requests	no
the opening enquiry and its reply	no
acknowledgements and the session control messages	no
parameter messages, and the replies and refusals to them	no
General MIDI System On / Off, sent or received	no
error reporting	no
tempo message (family 25)	yes

12.2 Which sockets carry what

The instrument has two sets of MIDI sockets. Only bulk-dump traffic is restricted to the first.

message	accepted on
bulk-dump data, continuation frames, handshake, acknowledgements	MIDI 1 only
dump requests	MIDI 1 only
parameter messages (families 2B, 2C)	MIDI 1 and MIDI 2
tempo message (family 25)	MIDI 1 and MIDI 2
General MIDI System On / Off	MIDI 1 and MIDI 2

MIDI channels are written with the socket set first: 1-01 to 1-16 and 2-01 to 2-16. No channel or device-number setting affects System Exclusive addressing — these messages carry no channel.

12.3 Memory protect

Two write-protect switches are consulted before an incoming message is acted on. A message they block never reaches the transfer machinery, and the instrument shows its own write-protection prompt rather than a System Exclusive error. A transfer that stops with that prompt on screen has not failed as a transfer.

Chapter 13

Errors

Transfer failures are reported on the SYSEX BULK DUMP screen. A System Exclusive message arriving while any other screen is displayed produces no visible error.

13.1 The messages

screen	meaning
COMPLETED!	the transfer finished
ERROR 40!	the message is addressed to a different product
ERROR 41!	reception failed
ERROR 42!	transmission failed
ERROR 21!	the destination is full

Not every failure is a 40, 41 or 42

When incoming data is larger than the destination area, the instrument reports ERROR 21! MEMORY FULL. Software that recognises only the three documented System Exclusive errors will mis-read a truncated restore as something else.

13.2 ERROR 40 — wrong identification

Raised when CMD is accepted but the bytes that follow it do not match the product. For a message addressed to the SX-WSA1R that means PC was not 04, or MD was neither 00 nor 01, or VER was not 11.

13.3 ERROR 41 — reception

The general reception failure. It covers:

- a message inside a session that does not begin F0 50;
- a status byte other than F7 arriving inside an open message;
- a framing, parity or overrun error on the MIDI input, or loss of active sensing;
- no reply within the reply window, including the acknowledgement that each data message must receive;

- a message longer than 254 bytes;
- an odd number of payload bytes;
- a continuation flag that is neither 00 nor 01;
- a checksum mismatch.

13.4 ERROR 42 — transmission

Raised when an outgoing transfer stalls inside the instrument: the send path waits for one block to be handed over internally, and reports ERROR 42 when that wait times out after about a second. In the ROM set described here this is its only cause.

ERROR 42 is not a report about the receiving device

The published manual describes ERROR 42 as an error during transmission, with the data not having been received correctly. That is not what the instrument tests. The condition it reports is internal to the sending machine and can arise with nothing connected to the MIDI output at all.

A genuine failure at the far end looks different. No acknowledgement within the reply window, a framing, parity or overrun error on the input, and a transfer abandoned after its three retries all raise ERROR 41. Software diagnosing a failed transfer should read ERROR 41, not ERROR 42, as the sign of a problem on the cable or at the receiver.

13.5 Interference from other traffic

While a bulk-dump session is open, unrelated System Exclusive traffic arriving on the MIDI input can set the error state, and will then be reported when the session ends. Keep other sources off the MIDI 1 input for the duration of a transfer.

Chapter 14

SX-WSA1 and SX-WSA1R

The SX-WSA1 keyboard and the SX-WSA1R rack module ship **the same ROMs** — not merely the same program built twice, but the same images — and speak the same System Exclusive protocol. Two features are nonetheless available on the keyboard and not on the rack, in both directions, and the two models put different bytes in the model triple. Nothing else in the System Exclusive implementation depends on which model it is: the framing, the checksum, the handshake, the error reporting, the parameter address space and every other bulk-dump category are identical.

	SX-WSA1 (keyboard)	SX-WSA1R (rack)
message framing, checksum, handshake	yes	yes
SOUND bulk dump	yes	yes
COMBINATION bulk dump	yes	yes
SYSTEM, PART & MIDI bulk dump	yes	yes
dump requests	yes	yes
General MIDI On / Off	yes	yes
MD transmitted	00	01
MD accepted	both	both
SEQUENCER bulk dump	yes	no
tempo message (family 25)	yes	no

On the rack, a SEQUENCER dump sends nothing and says nothing

The rack's SYSEX BULK DUMP menu has no SEQUENCER row: it is not drawn, and the soft key that would select it does nothing. The category can still be reached over MIDI, by a dump request addressed to 60 00 00, and as part of TOTAL KEYBOARD. In both cases nothing is transmitted — not even the end-of-category message that closes every other transfer. A librarian waiting for data will simply time out.

On the rack the transfer-progress screen likewise lists only SYS,PART&MIDI, SOUND and COMBINATION.

Technics' own guide points the same way: the photograph it prints of the SYSEX BULK DUMP menu, the one showing all five rows, is captioned (WSA1) — the marker the guide uses throughout for what belongs to the keyboard.

The tempo message is refused in both directions on the rack: it will neither act on one received nor send one.

Telling the two apart over MIDI

MD in anything the instrument transmits is 00 on the keyboard and 01 on the rack — the values Technics' published field table gives for the two models. Reading it back takes more than asking. HRQ and HRT are not an identity enquiry: they are the opening exchange of a bulk dump (chapter 5), they are acted on only inside it, and only one of the two produces a reply carrying MD at all. An HRQ is answered with an HRT, which does carry it. An HRT is answered with F0 50 23 7E F7 — an acknowledgement, with no model triple in it — so sending an HRT to ask what machine this is returns nothing to read.

Even the HRQ route needs the instrument to be showing the SYSEX BULK DUMP screen with the handshake state at zero, and answering it *opens a receive session*, which is a side effect a librarian probably did not want. Sent to an instrument on any other screen it produces silence — and not an ERR either, so nothing on the wire separates the wrong moment from the wrong machine or a dead cable.

What does work at rest is a *parameter* request (chapter 7) — not a dump request, which is DRQ too and is screen-gated like the handshake. Nothing in the parameter engine consults the panel, so a parameter DRQ is answered wherever the instrument happens to be, and the ITR that answers it carries MD substituted the same way. A request the instrument refuses is answered too, with F0 50 29 7E F7 — unlike 21 and 22, which are outside the set that error reply covers. So the parameter route reports a machine that is listening even when the address asked for is wrong, which makes it the better probe in both directions.

A third test is behavioural: request a SEQUENCER dump and see whether anything arrives.

Which machine is which

Because the ROMs are identical, the difference between the two models is not a firmware difference at all. Both behaviours are present in every machine, and a hardware input read once at power-on selects between them; the program names neither model. The identification does not depend on that, because Technics published the answer: the field table gives MD = 00 for the SX-WSA1 and 01 for the SX-WSA1R, and the machine that reads the second value is the one that transmits 01.

The same guide prints SEQUENCER : WSA1 ONLY beside the SEQUENCER row of its address map. Its user manual marks the whole sequencer chapter SEQUENCER (WSA1) and carries the same (WSA1) marker on fifty-nine further passages. A model with no sequencer has nothing to put in a SEQUENCER dump and no use for a tempo message, which is exactly the pair of System Exclusive functions that machine refuses.

Chapter 15

Relationship to other Technics instruments

The message engine described in this reference is not unique to the SX-WSA1R. At least two other Technics instruments implement the same protocol: the SX-KN5000 and the SX-KN1500.

15.1 What is shared

All three use the 50 identifier, the same message families, the same short control messages, the same payload encoding and the same checksum rule. The fixed messages are byte-for-byte identical:

F0 50 23 7E F7 F0 50 24 7E F7 F0 50 27 7E F7 F0 50 28 7E F7 F0 50 29 7E
F7 F0 50 2A 7E F7

The error mapping is also nearly common: of the thirty-four status values, thirty-two produce the same report on the SX-WSA1R and the SX-KN5000.

15.2 What distinguishes the products

PC, MD and VER together are the discriminator. A message whose three bytes do not match the receiving instrument is refused with ERROR 40.

instrument	PC MD VER
SX-WSA1 / SX-WSA1R	04 00 11, 04 01 11
SX-KN5000	01 28 12
SX-KN1500	01 24 11, 01 25 11, 01 26 11

The split is legible once the fields are named: PC = 04 is the WSA product category and 01 the keyboards, MD separates models within a category, and VER is the exclusive specification's own version — 2.1 for the WSA1 and the KN1500, 2.2 for the KN5000.

A librarian that supports more than one of these instruments needs only to vary those bytes and the dump region addresses; the framing, handshake, encoding and checksum are common code.

15.3 Dump regions differ

The dump region addresses are per-product. The SX-KN5000 and SX-KN1500 share eleven of their thirteen region and length pairs with each other, but the SX-WSA1R shares none of its eight with either — its categories are its own.

A fourth instrument's name sits in the signature table

The program holds its block signatures as literals, and immediately after WSA SOUND RAM S0 — its own — comes KN3000 SOUND RAM. No pointer in either program image reaches that second string, so nothing here shows it being used, and whether this instrument would accept an SX-KN3000 sound block is not established. The coverage of that negative is three-byte little-endian pointers across both 512 KB images; the string could still be reached by indexing from the first.

The SX-WSA1R does not accept the GS-style messages its siblings do

The SX-KN5000 and SX-KN1500 additionally accept a small set of messages in the 42 12 parameter form, including the sequence 42 12 40 00 7F 00 41 used elsewhere in the industry as a reset. **The SX-WSA1R accepts none of them.** Software that resets a KN-series instrument that way and assumes the SX-WSA1R will follow will find it does not respond.

Chapter 16

Worked examples

Three things in this format are easy to get subtly wrong: the seven-bit address encoding, the high-nibble-first payload, and which bytes the checksum covers. The examples below are complete messages, and every byte in them is computed rather than transcribed.

16.1 A parameter, written and read back

The example is part 0's parameter 03, a one-byte value, set to 100.

	message
write	F0 50 2C 04 00 11 00 20 03 00 00 01 06 04 00 41 F7
request	F0 50 2B 04 00 11 00 20 03 00 00 01 00 4C F7
reply	F0 50 2C 04 00 11 00 20 03 00 00 01 06 04 00 41 F7
part 7	F0 50 2C 04 00 11 00 27 03 00 00 01 06 04 00 3A F7
three-byte	F0 50 2C 04 00 11 00 20 00 00 00 03 01 02 00 00 04 00 00 45 F7

The value 100 is 64, which DT carries as the two bytes 06 04. The three bytes before it are SIZ, and the 00 after it is CN. SM closes the message.

16.2 A frame from a real instrument

The bytes below open the first data message of a dump recorded from an SX-WSA1R, a transfer of three categories in one session:

F0 50 2D 04 01 11 40 00 00 00 00 20 ...

Reading it by the rules of chapter 5: CMD is 2D BTR; 04 01 11 are PC, MD and VER, the middle one being the rack's; then ADR 40 00 00 and SIZ 00 00 20, which together name block 1 of SYSTEM, PART & MIDI. The length decodes to 32 source bytes and the frame carries 32 of them.

The payload decodes to 5A 5A 01 00 57 41 30 00..., the signature section 6.1 gives for that block.

That block is the one chapter 7 maps: a dump of this category can be read against table 7.1 directly, which is the cheapest way to confirm a parameter address without an instrument to hand.

Chapter 17

Scope of this edition

This reference covers message framing, the bulk-dump transfer protocol, the parameter address space, the sound and drum areas, the effects, the universal messages, error reporting, the differences between the two models, and the layout of the SOUND and COMBINATION blocks a dump carries.

17.1 Deliberately outside it

What a parameter does to the sound. Chapter 7 enumerates all 108 addresses of the 00 area with their name, length, bit mask, accepted values and position in a dump, and chapter 8 does the same for the sound areas. What a setting *does* is the user manuals' subject, not a protocol reference's.

The effect block diagrams. The guide draws the signal path of each of the 56 effects. Chapter 9 gives their parameters instead.

17.2 What the guide names but does not pin down

An implementation of this reference — a librarian that reads and writes every address in it — was built alongside this edition, and the count of what it could *not* turn into a control is the most honest measure of where the published tables stop. Three gaps, all of them the guide's rather than this document's, and each one a place where a reader must either measure the instrument or leave the parameter alone.

Bytes the guide addresses but does not break down. In NORMAL SOUND, 36 of 137 fields carry no bit layout and 20 carry no range; in DRUM SOUND, 29 of 99 and 8 respectively. The guide names each one and gives its address, and stops there. Several could be split by reading the prose beside them — TONE ON/OFF notes “two bits per tone, 4th 3rd 2nd 1st” — but a layout inferred from a sentence is a guess wearing a table's clothes, and chapter 8 prints the note instead.

Ranges given only as prose. A handful of parameters have a range the guide states in words rather than endpoints — “0.2 per cent per step” and the like. Those are printed as the note, not converted into a span, because the conversion would be a reading of the sentence rather than a transcription of a number.

Two gaps this edition closed by reading a column

Earlier drafts listed two more, and both turned out to be the guide stating something nobody had transcribed rather than the guide failing to state it.

The first was which conversion table each effect parameter uses. It is the DATA column of every DSP EFFECT page, and it is now given for all 435 parameters.

The second was the VALUE numbering where several fields share a byte. That is the VALUE column of the same pages, printed outright on every row, and it is now given for all 56 effects.

Both had been described as unstated on the strength of not having looked at the right part of the page. That is the same error this project has made repeatedly with these image-only manuals, and the remedy is the same: render the page before concluding it is silent.

17.3 Open, and what would close it

Other firmware. This reference describes one ROM set, and it can now name which. The instrument does not report an “operating system version” at all: it has a ROM VERSION screen that reports *three* ROMs separately, as WSA-A, WSA-C and WSA-D. Each program image ends with the linker’s own build stamp in its last sixteen bytes, and the screen’s code reads them — one routine twenty bytes from the caption checks the stamp’s tail, another loads from the same place. The set read here is

WSA-A 822 WSA-C 230 WSA-D 54

and a reader can compare all three against their own instrument in one screen. Three stamps for four images is not an omission: WSA-A spans two of them, which is why the screen lists three.

So “version 2” is a label the community archive puts on a file set, not a number the machine reports, and there is no single version for a reader to match. Describing a different ROM set is a different document and would need those ROMs; the three numbers above are what makes it possible to tell that you are holding one.